

Tests Web Service

Avec Postman

→ 16/03/2025

Agenda

Module	Date	Horaires	Salle
Module 1 Introduction à la méthodologie de test	04-mars	8h15-11h30	EF.202, Eiffel
Module 2 TDD / BDD	10-mars	8h15-11h30	SD.103, Bouygues
Module 3 Test Web Services : Postman	17-mars	8h15-11h30	Amphi e.068, Bouygues
Module 4 Automatisation Web Playwright	18-mars	13h45-17h	EF.110, Eiffel
Module 5 Automatisation App Mobile	21-mars	8h15-11h30	EF.202, Eiffel
Module 6 Test de performance : Gatling	24-mars	8h15-11h30	Amphi sc.071, Bouygues
Module 7 Accessibilité RGAA, Green, Data/IA	07-avr	8h15-11h30	TBD
Module 8 Evaluation	08-avr	13h45-17h	TBD

Logistique



Assiduité et émargement



Pauses



Interactivité et questions



Utilisation de laptops et smartphones



Evacuation

Sommaire

0. Introduction aux tests API
1. Installation et configuration Postman
2. Création des requêtes simples, ajout des vérifications
3. Enchaînement de plusieurs requêtes
4. Utilisation des variables
5. Utilisation des données CSV / JSON
6. Création d'une boucle
7. Cas pratique



0. Introduction aux tests API

Rappel ISTQB : niveaux de test et cas de test

Le syllabus de l'ISTQB définit 4 **niveaux de test** principaux :

1. Les tests composants : code fonctionne correctement de manière isolée
2. Les tests d'intégration : les différents éléments fonctionnent ensemble
3. Les tests système : le système complet fonctionne correctement
4. Les tests d'acceptation : répond aux attentes des utilisateurs

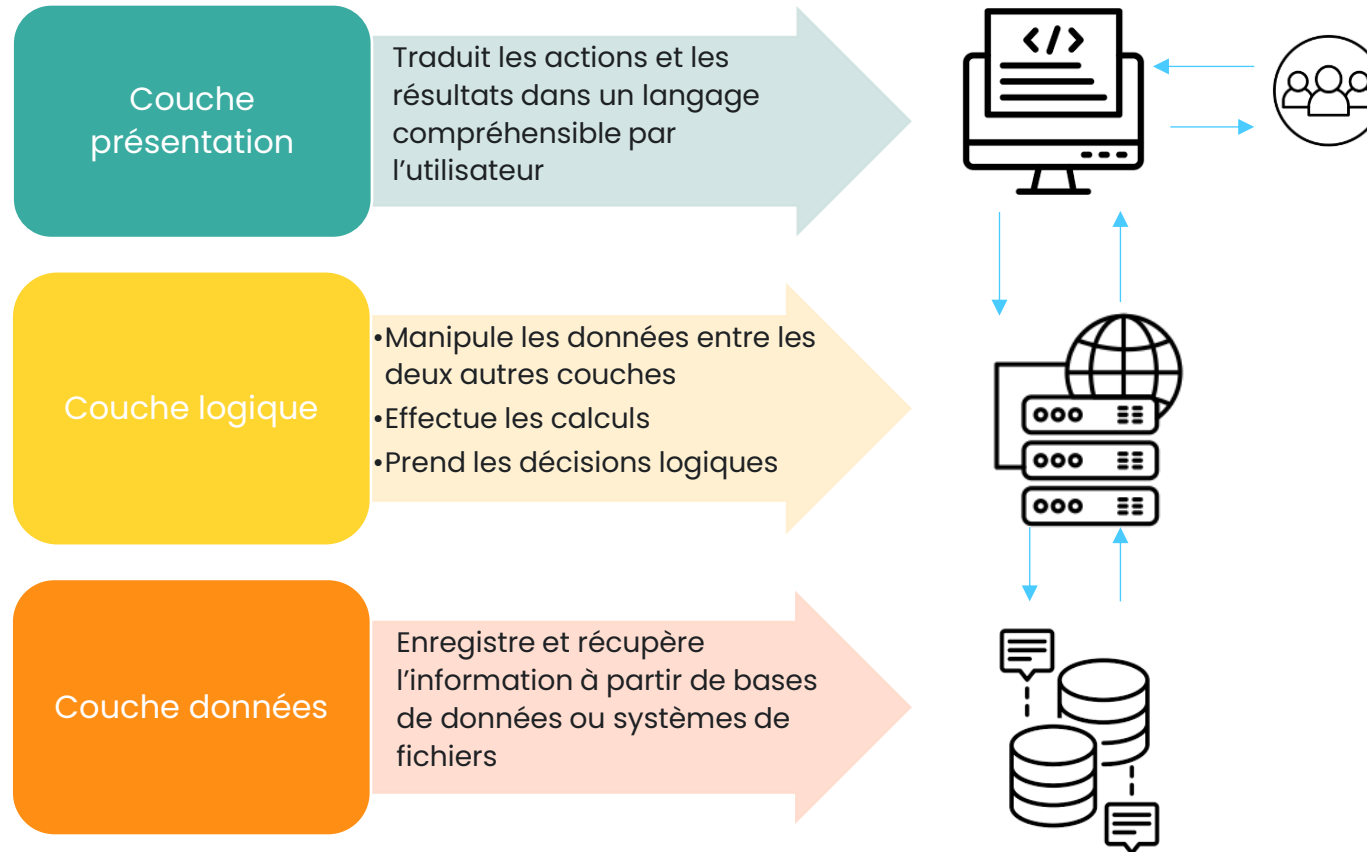
Conseil :

Penser à les différencier par les vérifications qui sont faites dans chaque niveau, et les risques couverts.

Définition d'un cas de test

Cas de test : un ensemble de valeurs d'entrée, de préconditions d'exécution, de résultats attendus et de post conditions d'exécution, développées pour un objectif ou une condition de tests particulier, tel qu'exécuter un chemin particulier d'un programme ou vérifier le respect d'une exigence spécifique [d'après IEEE 610]

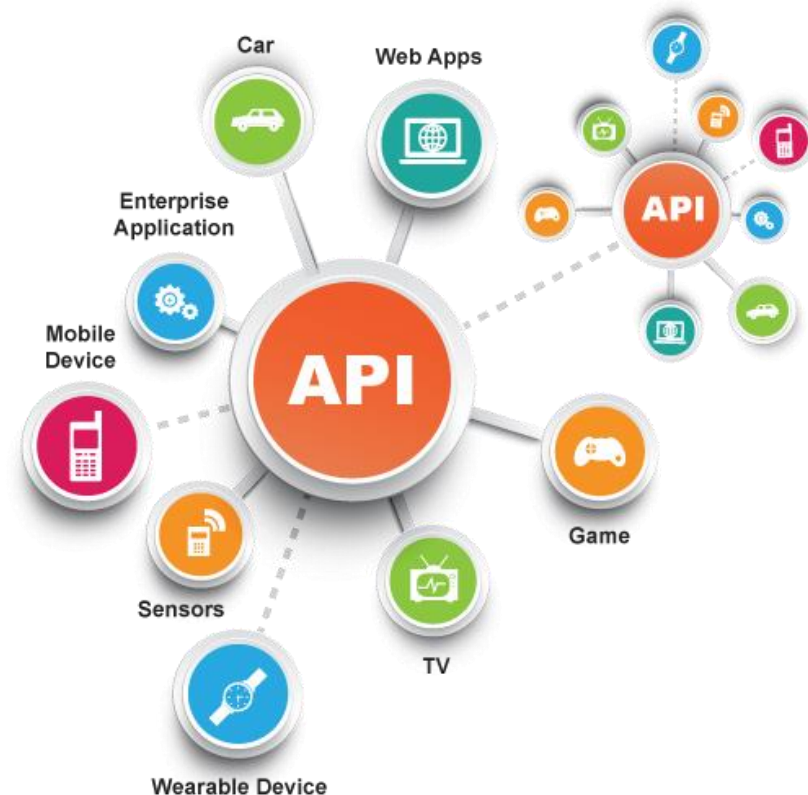
Architecture n-tiers



Les API – définition

- Les API (Application Programmable Interface) sont des ensembles de règles et standards qui étendent les fonctionnalités d'un composant ou d'une application.
- Ce sont des interfaces standardisées qui permettent à plusieurs applications d'échanger des informations.
- Une API définit en général :
 1. Les types de messages et de données que le composant peut accepter en entrée
 2. Les types et les contenus des réponses
- Les composants peuvent appartenir à la même application ou être fournis par des services tiers

Un web service est une implémentation Web d'une API.

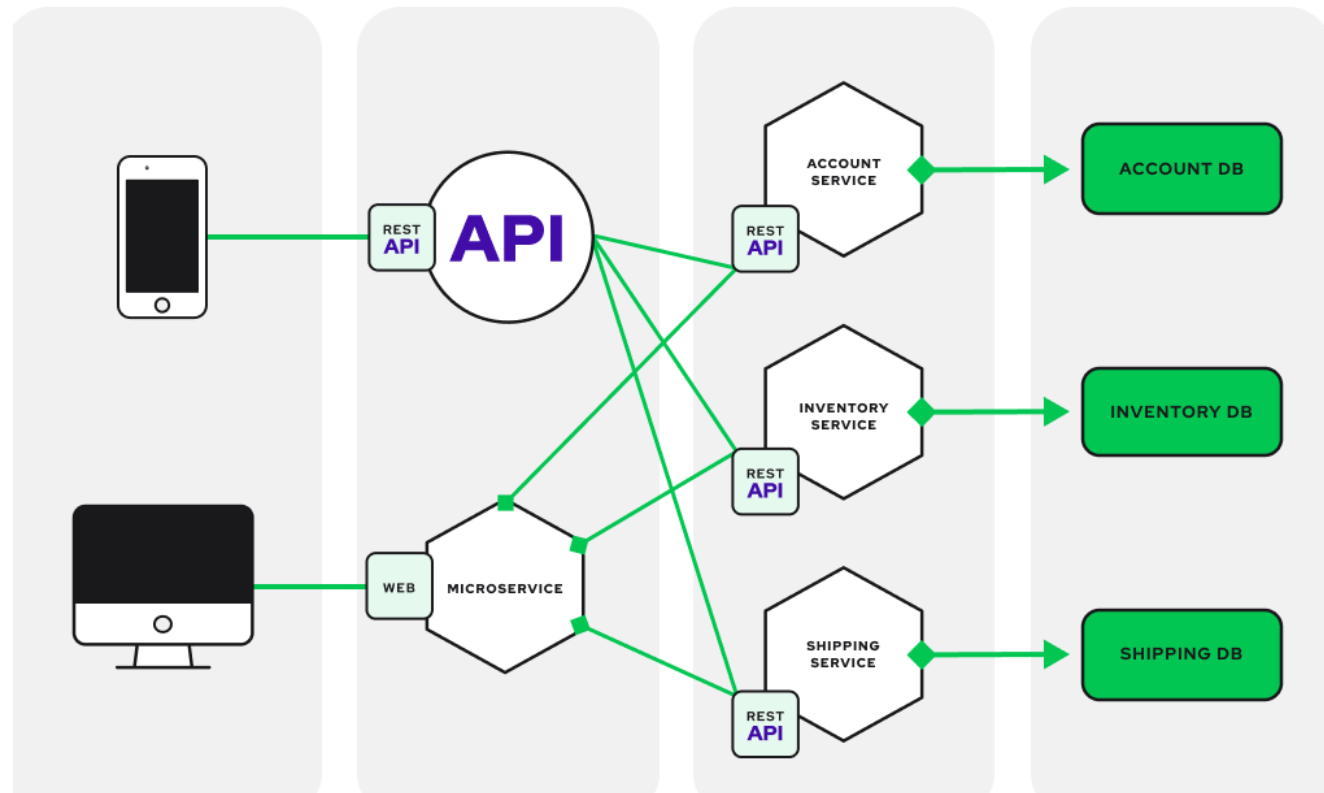


Architecture microservices

- Approche architecture et organisation du développement logiciel.
- Découpage en petits services indépendants qui communiquent via des API bien définies.
- Simplifient la mise à l'échelle et accélèrent le développement des applications
- Caractéristiques : Autonomie et spécialisation

Avantages

- Agilité
- Flexibilité pour le dimensionnement
- Facilité de déploiement
- Indépendance de la technologie
- Réutilisabilité du code
- Résilience



Web Services

Les web services reposent sur différents protocoles et technologies pour fonctionner

Protocoles

- **HTTP/HTTPS** : protocole de communication client-serveur développé pour le World Wide Web. Il est utilisé pour échanger toute sorte de données entre client HTTP et serveur HTTP.

Architecture

- **REST** : style d'architecture pour les environnements distribués décrivant comment construire des applications sur internet.
- **SOAP** : protocole standard de transmission de message décrit en XML. Il se présente comme une enveloppe pouvant être signée et pouvant contenir des données ou des pièces jointes. Il utilise le protocole HTTP et permet d'effectuer des appels de méthodes à distance.

Formats

- **XML** : Langage de balisage générique. On dit que ce langage est « extensible » car il permet de définir des espaces de noms, c'est-à-dire des langages ayant chacun leur propre vocabulaire et leur propre grammaire.
- **JSON** : JavaScript Objet Notation est un langage léger d'échange de données textuelles avec une syntaxe simple et une structure en arborescence
- **HTML** : Langage de balisage pour représenter les pages web.
- **WSDL** : Langage de description des Web services. Il s'agit de l'interface présentée aux utilisateurs qui indique comment utiliser un web service (types des données échangées, messages, etc.).

Langages

- **XPATH** : Langage permettant de requêter des portions dans un document XML.
- **XQUERY** : Langage permettant à la fois d'extraire des informations d'un document XML mais également de réaliser des calculs sur ces dernières.
- **JSONPATH** : syntaxe d'extraction des données JSON

Web Services

Protocole HTTP et requêtes

- HTTP est un protocole permettant la communication entre un client et un serveur. Une requête est toujours composée d'un appel (du client) et d'une réponse (du serveur). Voici un schéma résumant l'échange, les requêtes sont symbolisées par les flèches.

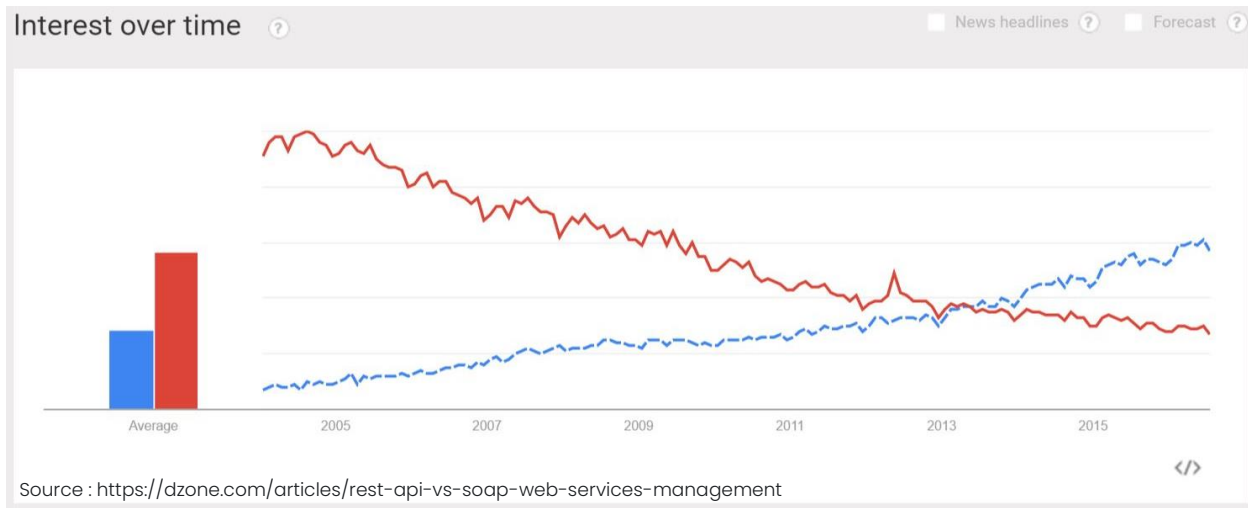


- Syntaxe d'une requête et d'une réponse HTTP:

Requête Client	Réponse Serveur
Ligne de commande • <u>Syntaxe</u> : Commande, URL, version de protocole. • <u>Exemple</u> : <code>GET /repertoire/page.html HTTP/1.1</code>	Ligne de statut • <u>Syntaxe</u> : version, Code-réponse, Texte-réponse. • <u>Exemple</u> : <code>HTTP/1.1 200 OK</code>
En-têtes de la requête • <u>Syntaxe</u> : « Nom entête :valeur ». • <u>Exemple</u> : <code>Host: www.site.com</code>	En-têtes de réponse. • <u>Syntaxe</u> : « Nom entête :valeur ». • <u>Exemple</u> : <code>Server: Apache/2.2.3</code>
Saut de ligne	Saut de ligne
Corps de la requête sert principalement à transmettre les formulaires HTML.	Corps de réponse Contient le contenu, dans le cas d'une page web le HTML par exemple.

SOAP vs REST

SOAP	REST
→ Orienté Services	→ Orienté Ressources (pages, images, etc.)
Avantages • Mécanisme d'appel de procédures distantes, on peut s'appuyer sur Soap dans un cadre contractuel. • Supporte le transactionnel • Formel, les méthodes et résultats sont spécifiés via des contrats (WSDL). • Fiable, mécanismes garantissant la fiabilité de la transmission et réception des données. • Soap est intégré dans la plupart des environnements de développement	Avantages • Léger et simple, messages courts et faciles à décoder pour le navigateur et serveur d'application • Auto-descriptif : navigation intuitive à travers les ressources. • Stateless: Adapté pour des opérations simples (consulter, supprimer, modifier, etc.) • Moins de dépendances sur les outils. • Architecture similaire aux sites internet.
Inconvénients • Soap est considéré comme trop complexe et verbeux pour des besoins ordinaires. • Nécessite des outils pour le développement	Inconvénients • Manque de complexité exigés par les transactions d'affaires • REST est incomplet pour fournir une solution fiable aux entreprises lors d'échanges complexes.



Exemple de messages SOAP

- CurrencyConverter est un service web Soap libre d'accès. Il propose une opération « ConversionRate » accessible sur 2 protocoles (Soap 1.1 et Soap 1.2).
- Voici une trace des échanges Soap effectués lors de la communication entre le serveur et le client :

→ Requête SOAP (client)

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:web="http://www.webserviceX.NET/">
  <soapenv:Header/>
  <soapenv:Body>
    <web:ConversionRate>
      <web:FromCurrency>EUR</web:FromCurrency>
      <web:ToCurrency>USD</web:ToCurrency>
    </web:ConversionRate>
  </soapenv:Body>
</soapenv:Envelope>
```

Données transmises

Soap Body (données échangées)

→ Réponse SOAP (serveur)

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <soap:Body>
    <ConversionRateResponse xmlns="http://www.webserviceX.NET/">
      <ConversionRateResult>1.0849</ConversionRateResult>
    </ConversionRateResponse>
  </soap:Body>
</soap:Envelope>
```

Données renvoyées

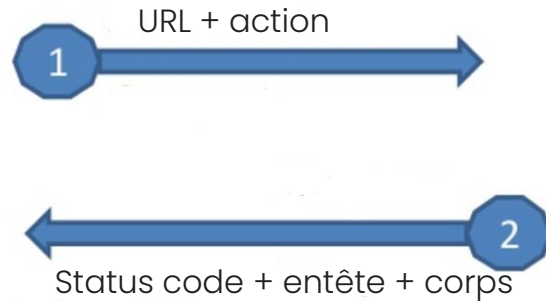
Soap Body (données échangées)

Exemple de message REST

Structure du message REST

Requête

GET https://eshop.com/books/14675



Réponse

HTTP status : 200 OK

*Allow : GET, HEAD, OPTIONS
content-type : application/json
vary : Accept*

```
{  
  "id": "14675",  
  "title": "Game of Thrones",  
  "author": "George R. R. Martin",  
  "isbn": "1-84356-028-3",  
  "availableQty": "4"  
}
```

Commandes et codes HTTP

Les commandes permettent de spécifier le type de requête et la méthode à utiliser. Voici une liste des commandes :

Commande	Description
GET	C'est la méthode la plus courante pour demander une ressource. Une requête GET est sans effet sur la ressource, il doit être possible de répéter la requête sans effet.
HEAD	Cette méthode ne demande que des informations sur la ressource, sans demander la ressource elle-même.
POST	Cette méthode doit être utilisée lorsqu'une requête ajoute une donnée ou modifie la ressource.
OPTIONS	Cette méthode permet d'obtenir les options de communication d'une ressource ou du serveur en général.
CONNECT	Cette méthode permet d'utiliser un proxy comme un tunnel de communication.
TRACE	Cette méthode demande au serveur de retourner ce qu'il a reçu, dans le but de tester et d'effectuer un diagnostic sur la connexion.
PUT	Cette méthode permet mettre à jour une ressource sur le serveur.
DELETE	Cette méthode permet de supprimer une ressource du serveur.

- Les codes permettent de renvoyer le résultat de l'exécution d'une requête par un serveur à un client :

Groupe	Exemple (Code, message et description)
Information	100 : Continue . Attend la suite de la requête
Succès	200 : OK . Requête traitée avec succès 201 : Created . Requête traitée avec succès avec création d'un document
Redirection	301 : Moved Permanently . Document déplacé de façon permanente 302 : Moved Temporarily . Document déplacé de façon temporaire
Erreur du client / du serveur web	401 : Unauthorized . Une authentification est nécessaire pour accéder à la ressource 403 : Forbidden . Le serveur a compris la requête, mais refuse de l'exécuter 404 : Not Found . Ressource non trouvée
Erreur du serveur / du serveur d'application	500 : Internal Server Error . Erreur interne du serveur 502 : Bad Gateway ou Proxy Error . Mauvaise réponse envoyée à un serveur intermédiaire par un autre serveur.





Savoir parler des API sans être un expert

Par Guillaume ADIN, Jean-Arnaud GAUTHUN et Yacine DECHMI

Définition API

API ou **Application Programming Interface** est une interface de programmation d'application proposant un ensemble de fonctions informatique normalisées qui permettent à une application d'exposer, communiquer et échanger du contenu avec d'autres applications.

Les 2 grands usages des API

Fourniture de Service

Exposer un service de bout en bout :

- Service d'authentification (google, facebook)
- Service de cartographie (google maps)
- Service de paiement (Apple Pay, PayPal, PayLib)
- Service intelligence artificielle (IBM Watson)

Partage de données

Partager des données en interne ou en externe

- Partager des données RH dans le SI
- Echanger des données entre deux applications
- Exposer des données à des partenaires

Les erreurs à éviter

Ne pas définir de format d'échange entre les API ou vers les API : complexité d'accès + maintenance à risque

Ouvrir l'accès à tous : Rendre visible des données confidentielles

Ne pas mettre de file d'attente : Surcharger le gestionnaire d'API

Ne pas déployer de cache* pour gérer les files d'attente de requêtes : Surcharger le gestionnaire d'API

Ne pas communiquer sur les modes d'accès aux API : Impossibilité d'utiliser les API

Ne pas communiquer sur les existences des API dans l'entreprise : Duplication des API

**Toutes les réponses à des requêtes identiques sur un laps de temps (à définir) sont les mêmes*

L'API management

La gestion des API permet de gérer le peuplement des API dans SI, et permet de :

- Gérer le dispatch sur les API (empêcher les plus demandées soient saturées) – Passerelle API
- Mettre les API à disposition de façon sécurisée
- Gérer le cycle de vie des APIs
- Fournir de l'information sur l'utilisation de l'API
- Monétiser l'utilisation des API

Matrice d'évaluation by onepoint

Gouvernance

- Stratégie
- Gouvernance
- Considération juridique

API Engineering

- Design
- Implémentation
- **Qualité**
- Infrastructure

Business Model

- Chaine de valeur
- Marketing
- Documentation

Cyber Sécurité

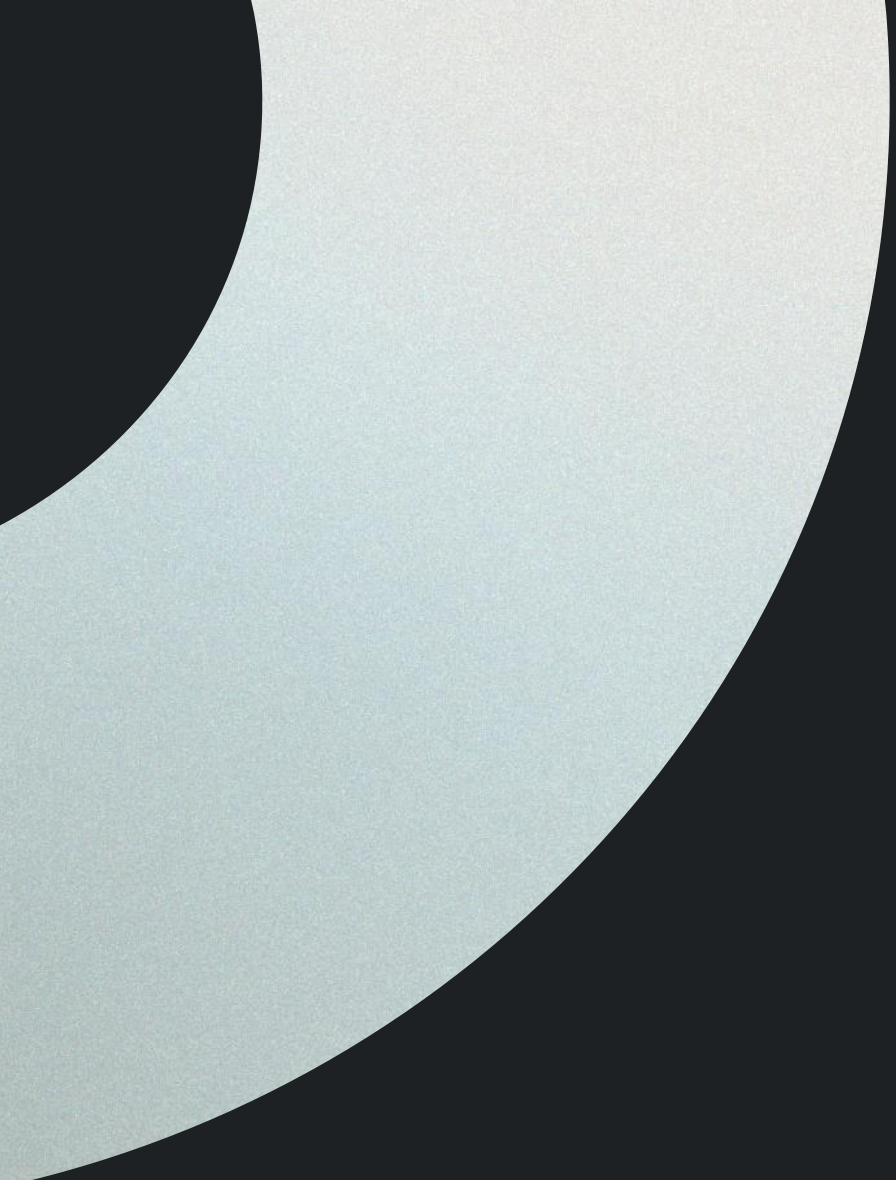
- Règles de sécurité
- Traçabilité
- Audibilité
- Sécurisation de l'API

Support

- Opérer les APIs
- Support offert aux développeurs

Un exemple concret





1. Installation et configuration Postman

Pourquoi utiliser Postman ?



- Un **outil gratuit** de test des webservices principalement en REST.
- **Accessibilité et convivialité**
- Peut-être implémenté sur multiple système d'exploitation : Windows, Mac OS, Linux.
- Une large communauté
- Les appels d'API sans avoir à coder.
- Fonctionnalités riches.
- Largement répondu.

1. Poste Windows 10/11
2. Compte Postman actif : <https://identity.getpostman.com/signup>
3. Télécharger l'application et l'installer : <https://www.postman.com/downloads/>
4. Télécharger l'application FlightApi.zip et dézipper le contenu dans un dossier de travail
https://github.com/imhah-hassan/Postman_training/blob/master/Demo%20App/FlightApi.zip



Installation

Télécharger Postman : <https://www.postman.com/downloads/>

The Postman app

Download the app to get started with the Postman API Platform.

 Windows 64-bit

 Postman-win64-Se....exe ^



Vous pouvez créer un nouveau compte

Create an account or sign in

Create Free Account

Sign in

Connectez vous avec le compte que vous venez de créer !



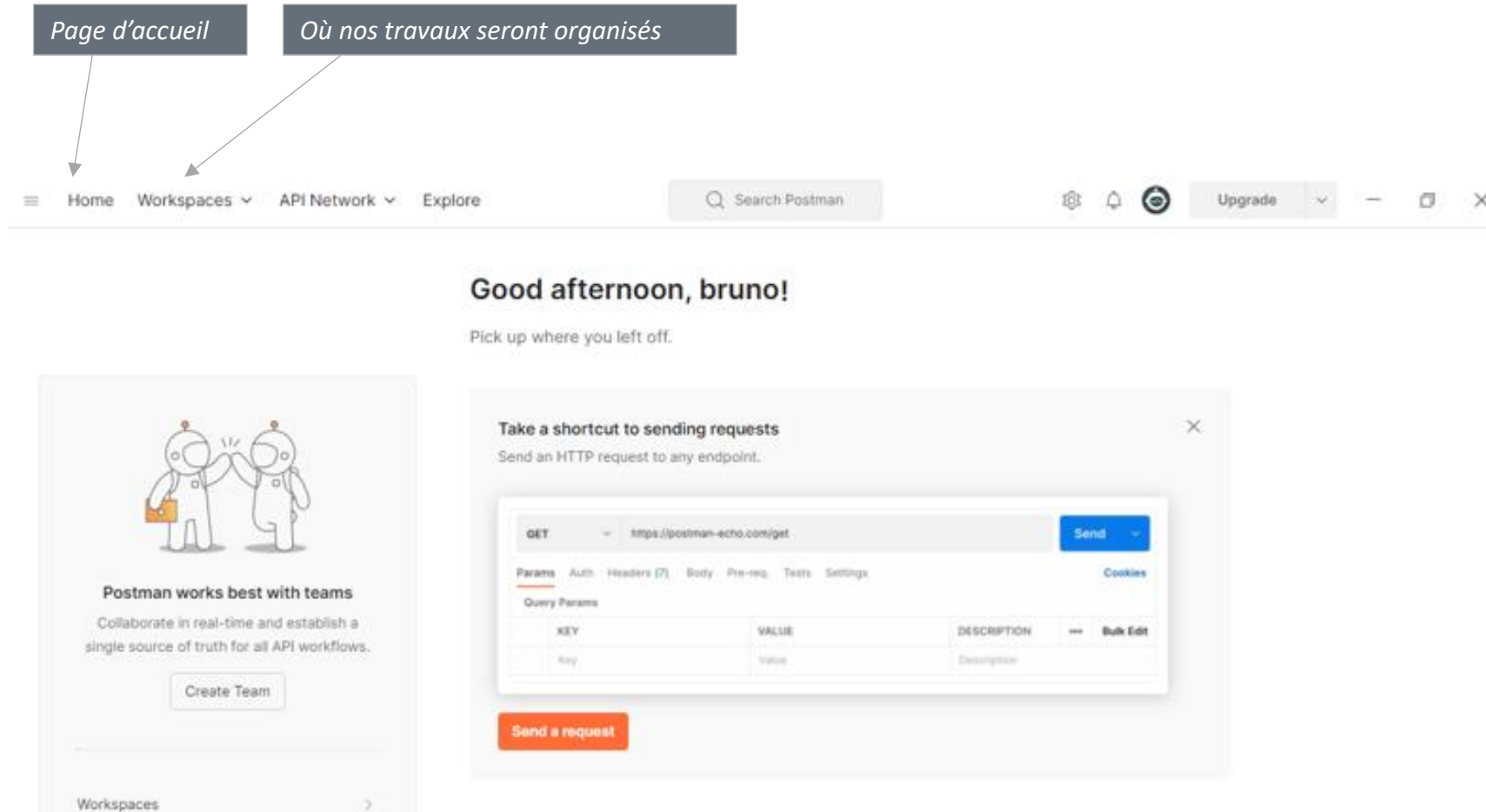
Sign In [Create Account](#) instead?

Email or Username

Password

☒ Stay signed in for 30 days [Forgot Password?](#)

La page d'accueil



Création du Workspace

Créer un espace de travail pour chacun de vos projets

Création du nouveau Workspace

1

Retrouver ici la liste de vos espaces de travail

Saisir le nom du projet

2

Choisir Personal

3

Valider

4

Create workspace

Name

Formation Postman

Summary

Add a brief summary about this workspace.

Visibility

Determines who can access this workspace.

☒ Personal
Only you can access

☐ Private
Only invited team members can access

☐ Team
All team members can access

☐ Partner NEW
Only invited partners and team members can access

☐ Public
Everyone can view

Create Workspace Cancel

Création d'une collection

Créer une collection (ensemble de requêtes présentant avec une cohérence)

Création d'une nouvelle collection

Saisir le nom de la collection

Create a collection for your requests

A collection lets you group related requests and easily set common authorization, tests, scripts, and variables for all requests in it.

Create Collection

Exercise - Requetes Simple

New Collection

This collection is empty. [Add a request](#) to start working.

Auth Pre-req. Tests Variables Runs

This authorization method will be used for every request in this collection. You can override this by specifying one in the request.

Type No Auth

Ou clic droit sur la collection, et choisir Rename

Structure

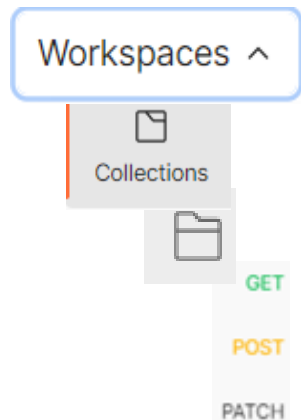
Organisation des requêtes :

- Chaque Workspace peut contenir plusieurs collections
- Chaque collection peut être organisée avec des dossiers
- Les requêtes peuvent être ajoutées dans un dossier ou directement dans la collection

Organisation des variables

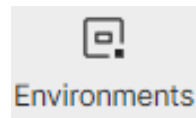
- Les variables peuvent être globales ou liées à une collection.
- Les variables peuvent aussi être organisées dans les environnements pour simplifier la maintenance.

Requêtes



Variables

Globals



Exercice 0 : Préparation

- 0. Préparation
 - 0.1. Création d'un workspace
 - 0.2. Création de collection
 - 0.3. Importer / Exporter une collection
 - 0.4. Création d'une requête API
 - 0.5. Liste des requêtes FlightsAPI utilisées

Documentation de l'API Flights :

<https://app.swaggerhub.com/apis/imhah-hassan/FligthApi/Mars2023>

2. Création des requêtes simples, ajout des vérifications



Requête Rest dans Postman

GET ⌵ `https://api.taiga.io/api/v1/issues?project=404046`

Params • Authorization Headers (8) Body Pre-request Script Tests Settings

Headers 7 hidden

	KEY	VALUE
☒	Content-Type	application/json

Params • Authorization Headers (8) Body Pre-request Script

Query Params

	KEY	VALUE
☒	project	404046

Params Authorization Headers (10) **Body •**

☒ none ☐ form-data ☐ x-www-form-urlencoded

```
1 {  
2   "password": "Hassan$2022",  
3   "type": "normal",  
4   "username": "h.imhah"  
5 }
```

Params **Authorization** Headers (10)

Type Inherit a...

The authorization header will be automatically generated when you send the request. Learn more about [authorization](#)

Params Authorization Headers (10) Body • Pre-request Script **Tests •**

```
1 pm.test("Status code is 200", function () {  
2   pm.response.to.have.status(200);  
3 });  
4  
5 pm.test("Body matches auth_token string", function () {  
6   pm.expect(pm.response.text()).to.include("auth_token");  
7 });  
8  
9 pm.test("Verify user name", function () {  
10   var jsonData = pm.response.json();  
11   pm.expect(jsonData.username).to.eql("h.imhah");  
12 });
```

GET ⊗ ⌵ ht

GET

POST

PUT

PATCH

DELETE

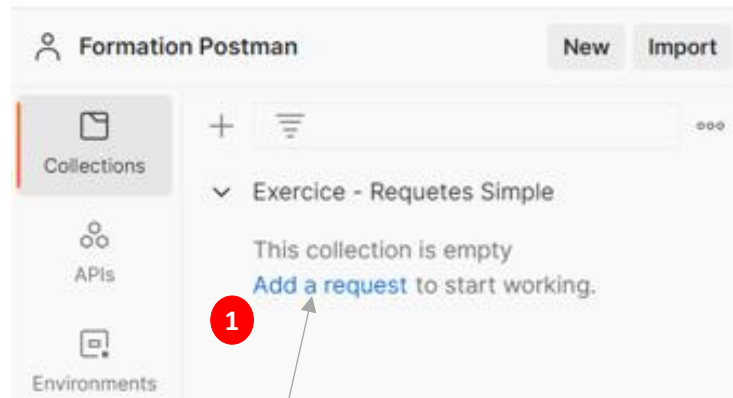
COPY

Params Authorization Headers (11) Body • **Pre-request Script •**

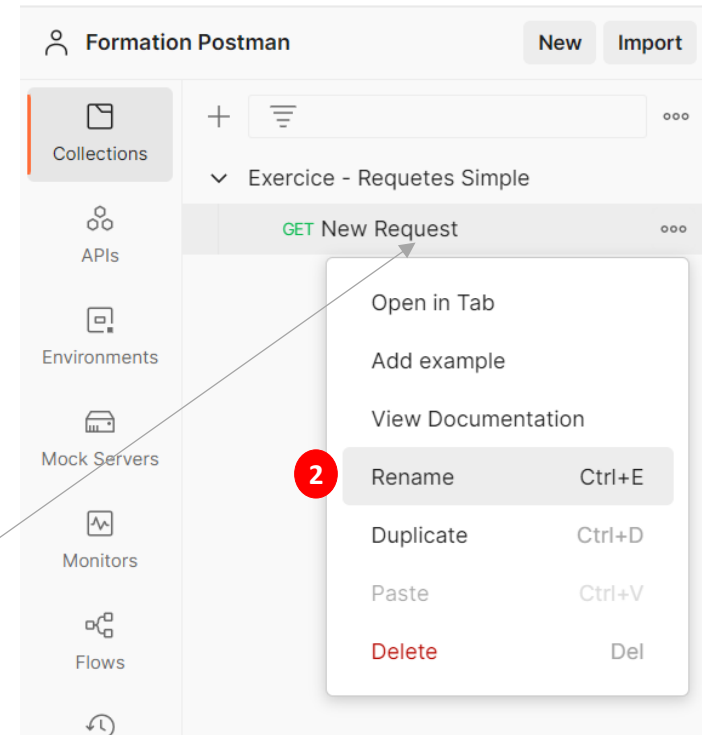
```
1 var issuesCount = pm.environment.get ("issuesCount")  
2 console.log (pm.environment.get ("issuesCount"));  
3  
4 if (issuesCount > 1) {  
5   postman.setNextRequest ("issues");  
6 }
```

Création d'une requête API

Lancer une requête pour lire les informations d'un vol via son numéro.

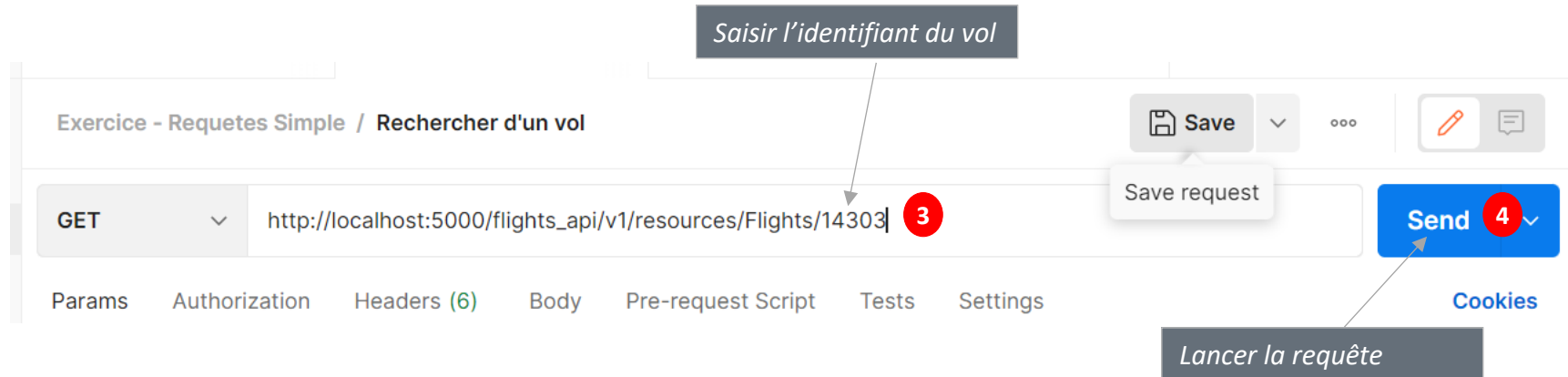
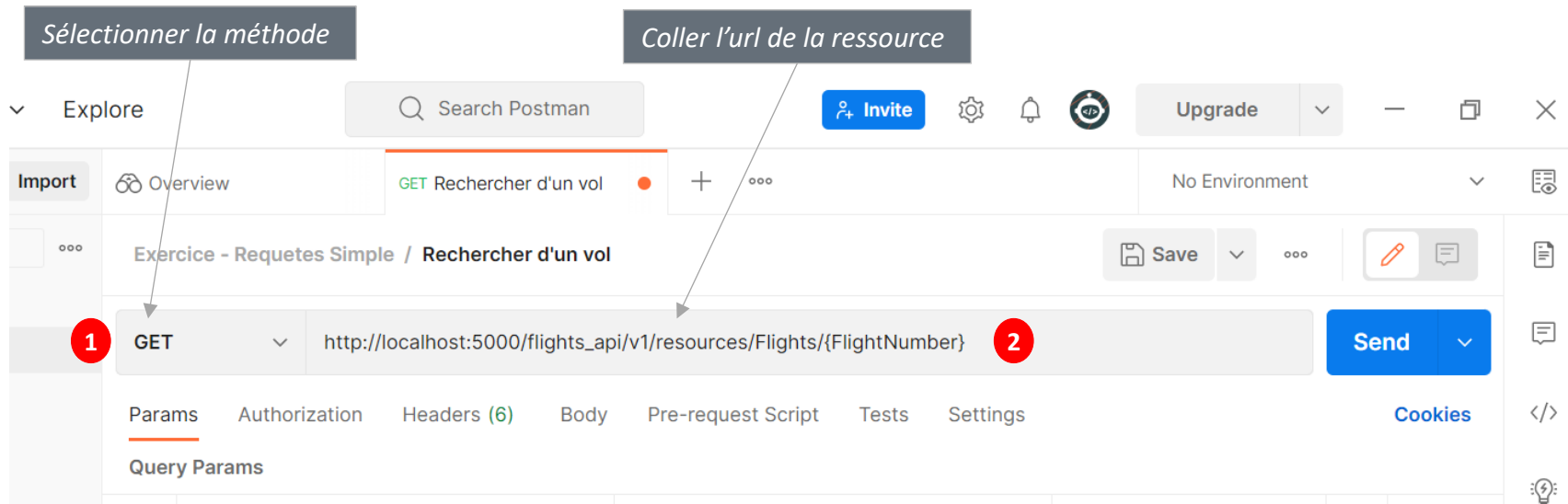


Création d'une requête (*Add a request*)



Clic droit sur la requête,
puis choisir *Rename*

Création d'une requête simple (GET)



N'oublier pas de sauvegarder (**Save**)(ou **CRL + S**) avant de lancer votre requête (**Send**)

Résultat d'une requête simple (GET)

En retour une réponse avec un code 200 (ok) et les informations en format JSON du vol.

The screenshot shows the Postman interface with a GET request to `http://localhost:5000/flights_api/v1/resources/Flights/14303`. The response status is **200 OK** with a response time of 81 ms and a size of 360 B. The response body is displayed in JSON format, showing flight details for flight 14303.

KEY	VALUE	DESCRIPTION
Key	Value	Description

```
1  {
2    "Airline": "NW",
3    "ArrivalCity": "Frankfurt",
4    "ArrivalTime": "07:41 PM",
5    "DepartureCity": "Denver",
6    "DepartureTime": "06:57 PM",
7    "FlightNumber": 14303,
8    "Price": 100.8,
9    "SeatsAvailable": 250,
10   "DayOfWeek": "Friday"
11 }
```

Réponse au format json

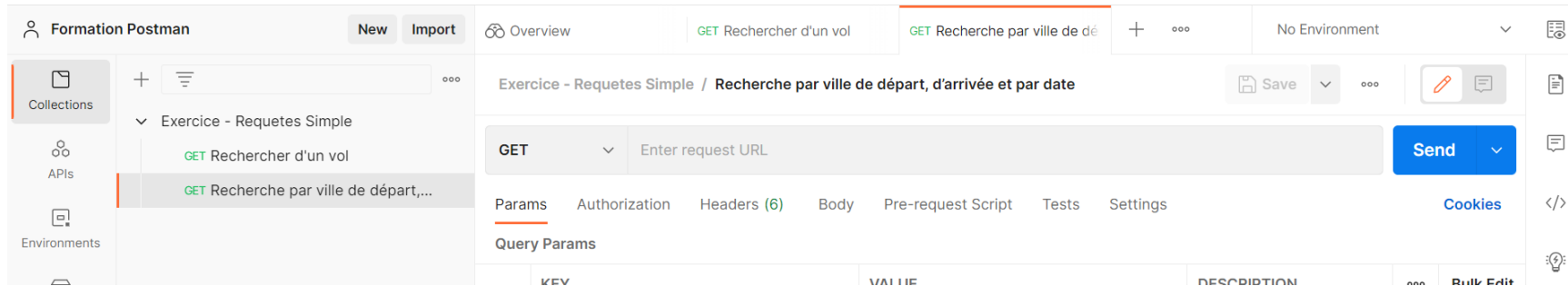
HTTP Status Codes



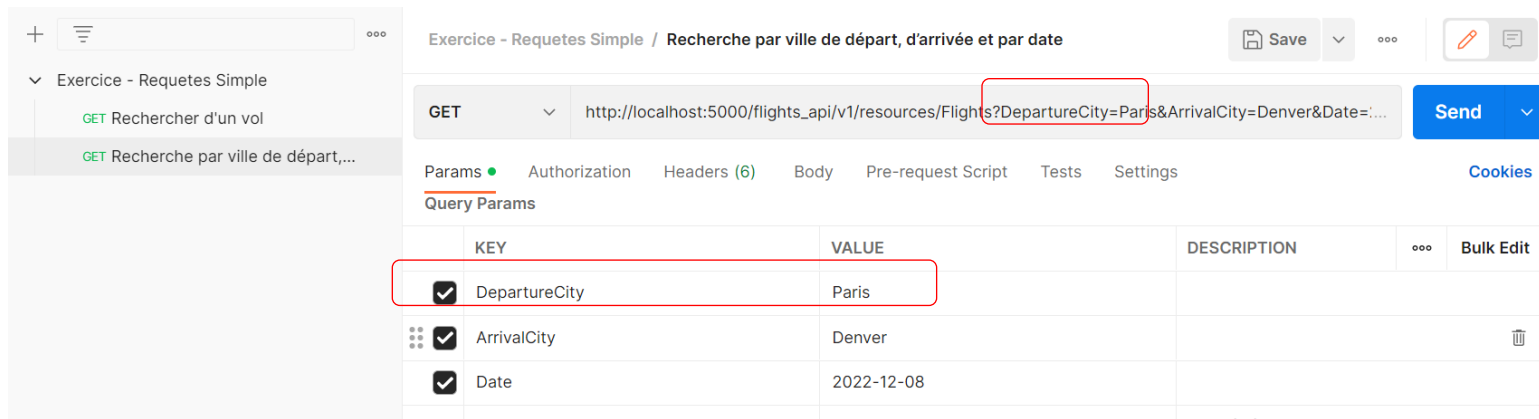
Création d'une requête (GET)

Rechercher un vol par ville de départ, ville d'arrivée et par date

Faire un clic droit sur la collection (Exercice – Requêtes Simple), puis choisir le menu **Add Request**
Renommer la requête



Sélectionner la méthode (GET) et coller l'url de la ressource

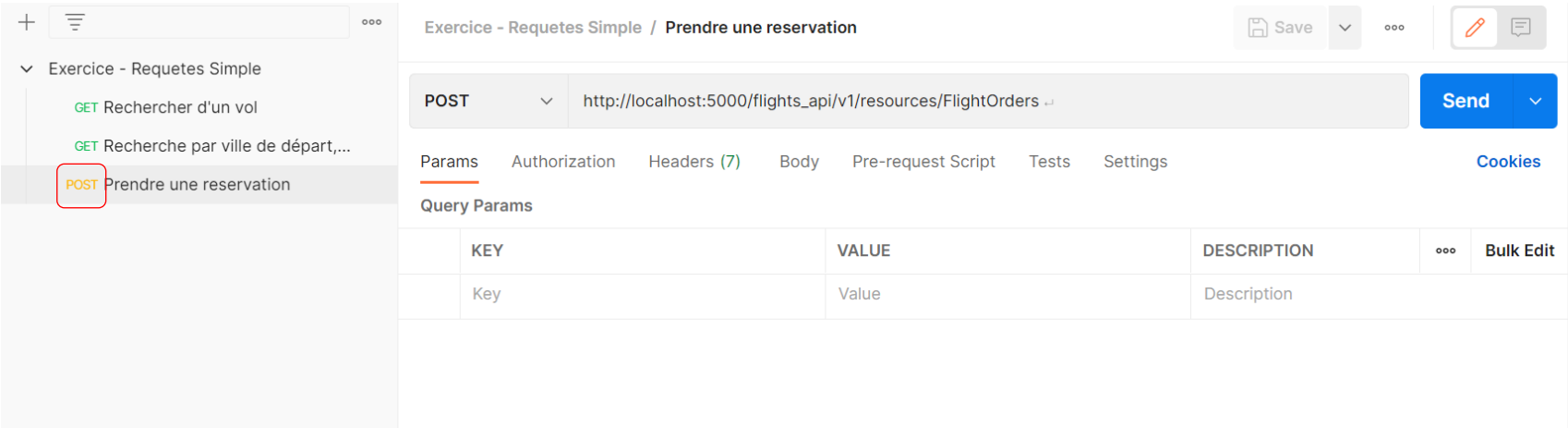


Pour modifier une requête vous pouvez utiliser l'onglet **Params** au lieu de modifier URL,
Par exemple, remplacer la valeur Paris par la valeur London pour le paramètre **DepartureCity**

Création d'une requête (POST)

Prendre une réservation

Créer une requête et la nommer. Sélectionner la méthode (POST) et coller l'url de la ressource



Après la sauvegarde, la requête est tagguée comme étant un requête de type POST

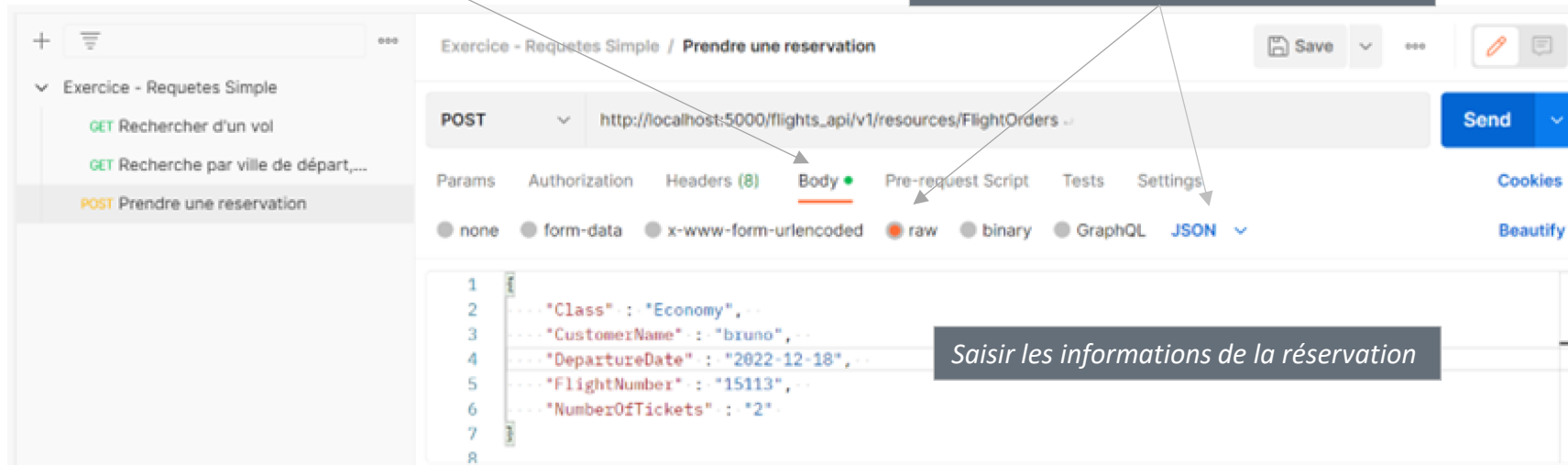


Création d'une requête (POST)

Envoyer les informations de la nouvelle réservation

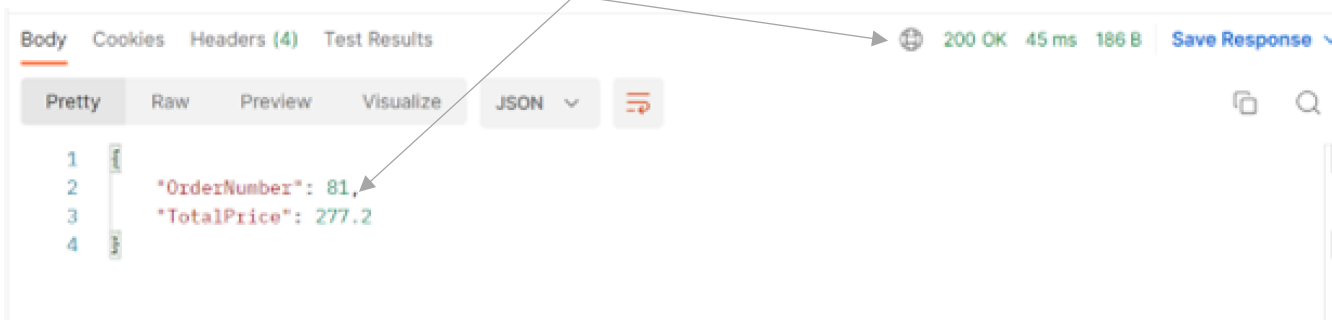
Cliquer sur l'onglet **Body** pour accéder aux formats disponibles

Choisir le format **Raw** et préciser **JSON**



Saisir les informations de la réservation

Enregistrer la requête et envoyer la : Réponse est retournée avec l'identifiant de la réservation



Création d'une requête (PATCH)

Modifier une réservation existante

The screenshot shows a REST client interface with a sidebar on the left containing a list of requests under the heading "Exercice - Requetes Simple". The selected request is "PATCH Modifier une reservation".

The main area displays the details of the selected request:

- Method:** PATCH
- URL:** `http://localhost:5000/flights_api/v1/resources/FlightOrders/81`
- Body:** A JSON object with the following fields:

```
{  "Class": "Economy",  "CustomerName": "bruno",  "DepartureDate": "2022-12-18",  "FlightNumber": "15113",  "NumberOfTickets": "5"}
```

The response section at the bottom shows the result of the request:

- Status:** 200 OK
- Time:** 40 ms
- Size:** 314 B
- Body:** A JSON object with the following fields:

```
{  "Class": "Economy",  "CustomerName": "bruno",  "DepartureDate": "2022-12-18 18:15",  "FlightNumber": 15113,  "NumberOfTickets": 5,  "OrderNumber": 81,  "TicketId": "2022-12-18 18:15"}
```

Annotations on the image:

- "Appeler la réservation à modifier (81)" points to the URL.
- "Sélectionner la méthode" points to the PATCH method.
- "Saisir les informations de la réservation dont le nouveau nombre de tickets" points to the JSON body.
- "Nouvelle valeur prise en compte" points to the updated "NumberOfTickets" value of 5 in the response.

Création d'une requête (DELETE)

Supprimer une réservation existante

Appeler la réservation à modifier (81)

Sélectionner la méthode

DELETE http://localhost:5000/flights_api/v1/resources/FlightOrders/81

Params Authorization Headers (6) Body Pre-request Script Tests Settings Cookies

KEY	VALUE	DESCRIPTION	...	Bulk Edit
Key	Value	Description		

Body Cookies Headers (4) Test Results 200 OK 60 ms 174 B Save Response

```
1 {
2   "true": "Order deleted 81 "
3 }
```

Confirmation de la suppression

The screenshot shows a REST client interface with a sidebar on the left containing a list of requests. The main panel displays a DELETE request to 'http://localhost:5000/flights_api/v1/resources/FlightOrders/81'. The response is a 200 OK status with a body containing the JSON object: { "true": "Order deleted 81 " }. Annotations with arrows point to the method selection, the URL, the response status, and the response body.

Consulter la log d'exécution

Consultation des logs d'exécution en cliquant sur Console

The screenshot displays a REST client interface. On the left, a sidebar shows 'Collections' with a tree view for 'Exercice - Requetes Simple'. The main panel shows a 'DELETE' request to 'http://localhost:5000/flights_api/v1/resources/FlightOrders/81'. The response is '200 OK' with a body of 'true: "Order deleted 81"'. Below the main panel, a 'Console' tab is selected, showing a list of executed requests. A red box highlights the 'Console' tab in the sidebar. An arrow points from a text box at the bottom left to the 'Console' tab. Another arrow points from a text box at the bottom right to the 'Clear' button in the console header.

Exercise - Requetes Simple / Supprimer une reservation

DELETE http://localhost:5000/flights_api/v1/resources/FlightOrders/81

Params Authorization Headers (6) Body Pre-request Script Tests Settings

Body Cookies Headers (4) Test Results

200 OK 60 ms 174 B Save Response

1 "true": "Order deleted 81"

2

Console

▶ GET http://localhost:5000/flights_api/v1/resources/Flights/14303 200 81 ms

▶ POST http://localhost:5000/flights_api/v1/resources/Flights/14303 405 9 ms

▶ GET http://localhost:5000/flights_api/v1/resources/Flights?DepartureCity=Paris&ArrivalCity=Denver&Date=2021-12-08 200 90 ms

▶ GET http://localhost:5000/flights_api/v1/resources/Flights?DepartureCity=Paris&ArrivalCity=Denver&Date=2022-12-08 200 20 ms

▶ GET http://localhost:5000/flights_api/v1/resources/Flights?DepartureCity=London&ArrivalCity=Denver&Date=2022-12-08 200 12 ms

▶ POST http://localhost:5000/flights_api/v1/resources/FlightOrders%0A 500 127 ms

▶ POST http://localhost:5000/flights_api/v1/resources/FlightOrders%0A 200 45 ms

▶ PATCH http://localhost:5000/flights_api/v1/resources/FlightOrders/81%0A 200 41 ms

▶ PATCH http://localhost:5000/flights_api/v1/resources/FlightOrders/81%0A 200 44 ms

▶ PATCH http://localhost:5000/flights_api/v1/resources/FlightOrders/81%0A 200 40 ms

▶ DELETE http://localhost:5000/flights_api/v1/resources/FlightsOrders/81 404 8 ms

Clear

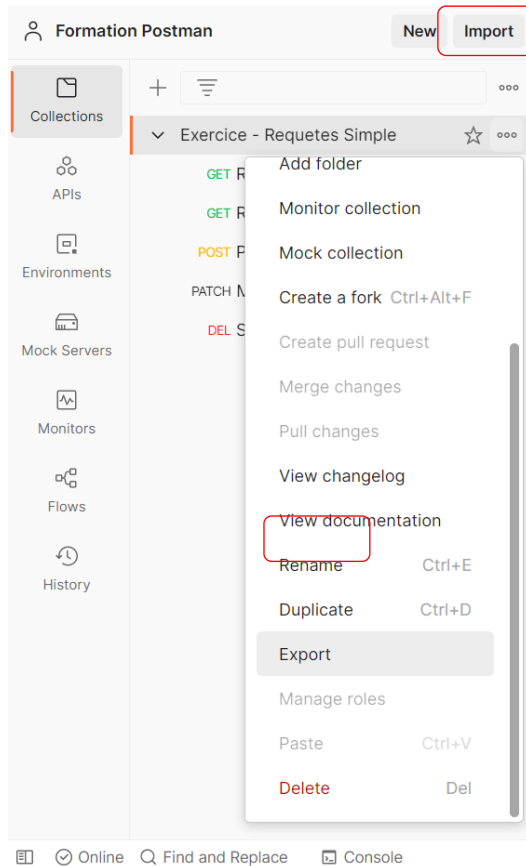
Pour effacer le contenu cliquer sur **Clear**.

Pour afficher le détail, cliquer sur la requête

Importer / Exporter une collection

Pour exporter une collection, faire un clic droit sur la collection qu'on souhaite exporter,
Cliquer sur le bouton **Export** dans la popup qui s'affiche,
Choisir l'emplacement du fichier JSON et confirmer l'enregistrement..

Pour l'import d'une collection, cliquer sur le bouton « Import », puis naviguer jusqu'à votre fichier de collection

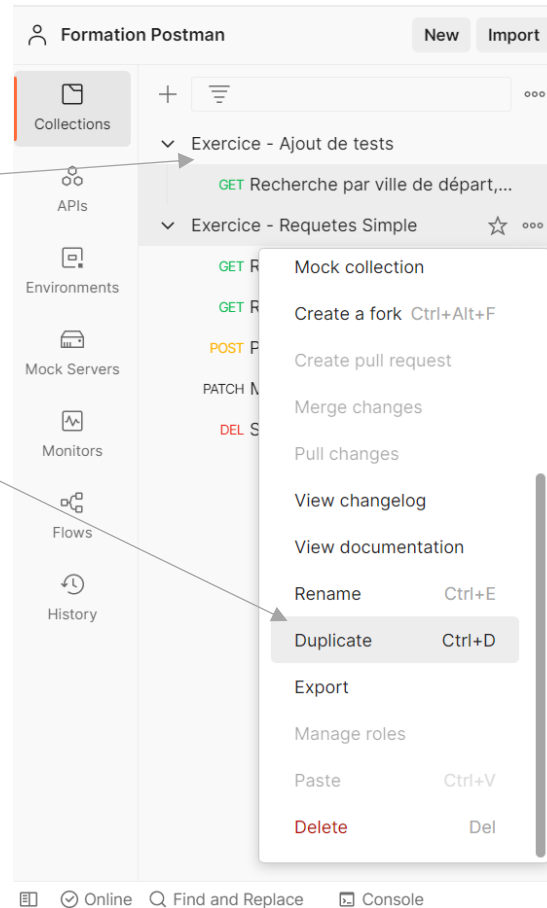


Vérifier un code retour

Préparation d'une nouvelle collection

Création d'une nouvelle collection

*Dupliquer la requête :
Recherche par ville de départ, d'arrivée et par date,
Puis déplacer la vers la nouvelle collection*



Note : On peut également dupliquer une collection

Vérifier un code retour

Mise au point d'un premier test

The screenshot shows the Postman interface with a REST client test named "Recherche par ville de départ, d'arrivée et par date Copy". The URL is `http://localhost:5000/flights_api/v1/resources/Flights?DepartureCity=Paris&ArrivalCity=Denver&Date=...`. The "Tests" tab is active, showing a JavaScript test script:

```
1 pm.test("Status code is 200", function () {  
2   pm.response.to.have.status(200);  
3 });
```

Annotations on the image:

- 1: Points to the "Tests" tab.
- 2: Points to the "Status code: Code is 200" snippet in the "Snippets" panel.
- 3: Points to the JavaScript test script in the "Tests" tab.

Callouts:

- "Cliquez sur l'onglet Tests" (Click on the Tests tab) points to the "Tests" tab.
- "Cliquez sur le Snippets Status code: Code is 200" (Click on the Status code: Code is 200 snippet) points to the snippet in the "Snippets" panel.
- "Le code JavaScript est ajouté automatiquement" (The JavaScript code is added automatically) points to the test script in the "Tests" tab.

The "Body" tab is selected, showing the response body in "Pretty" format:

```
{  
  "Airline": "AF",  
  "ArrivalCity": "Denver",  
  "ArrivalTime": "10:30 AM",  
  "DepartureCity": "Paris",  
  "DepartureTime": "08:00 AM",  
  "FlightNumber": 17105,  
  "Price": 139.47,  
  "SeatsAvailable": 250,  
  "DayOfWeek": "Thursday",  
  "Airline": "AF",  
  "ArrivalCity": "Denver",  
  "ArrivalTime": "12:54 PM",  
  "DepartureCity": "Paris",  
  "DepartureTime": "10:24 AM",  
  "FlightNumber": 17109,  
  "Price": 143.47,  
  "SeatsAvailable": 250,  
  "DayOfWeek": "Thursday",  
  "Airline": "AF",  
  "ArrivalCity": "Denver",  
  "ArrivalTime": "01:48 PM",  
  "DepartureCity": "Paris",  
  "DepartureTime": "11:30 AM",  
  "FlightNumber": 17113,  
  "Price": 143.47,  
  "SeatsAvailable": 250,  
  "DayOfWeek": "Thursday"  
}
```

The status bar at the bottom shows "200 OK 24 ms 1.4 KB".

Notes : N'oubliez pas de sauvegarder

Un snippet (« **extrait** ») est une partie d'un code source pouvant être copié-collé dans un modèle plus large.

Vérifier un code retour

Exécutions de la requête suivi de la vérification

1 - Cas passant :

+

...

Exercice - Ajout de tests

GET Recherche par ville de départ,...

Exercice - Requetes Simple

GET Rechercher d'un vol

GET Recherche par ville de départ,...

POST Prendre une reservation

PATCH Modifier une reservation

DEL Supprimer une reservation

Exercice - Ajout de tests / Recherche par ville de départ, d'arrivée et par date Copy

Save

...

GET

http://localhost:5000/flights_api/v1/resources/Flights?DepartureCity=Paris&ArrivalCity=Denver&Date=...

Send

Params

Authorization

Headers (6)

Body

Pre-request Script

Tests

Settings

Cookies

1

2

3

pm.test("Status code is 200", function () {

pm.response.to.have.status(200);

});

Test scripts are written in JavaScript, and are run after the response is received. Learn more about tests scripts

Body

Cookies

Headers (4)

Test Results (1/1)

200 OK

14 ms

1.4 KB

Save Response

All

Passed

Skipped

Failed

PASS

Status code is 200

La vérification est au statut **Pass**

2 - En modifiant le nom de la ville par une ville non référencée, le test devient non passant :

+

...

Exercice - Ajout de tests

GET Recherche par ville de départ,...

Exercice - Requetes Simple

GET Rechercher d'un vol

GET Recherche par ville de départ,...

POST Prendre une reservation

PATCH Modifier une reservation

DEL Supprimer une reservation

Exercice - Ajout de tests / Recherche par ville de départ, d'arrivée et par date Copy

Save

...

GET

http://localhost:5000/flights_api/v1/resources/Flights?DepartureCity=Parisparis&ArrivalCity=Denver&D...

Send

Params

Authorization

Headers (6)

Body

Pre-request Script

Tests

Settings

Cookies

KEY	VALUE	DESCRIPTION	...	Bulk Edit
☒	DepartureCity	Parisparis		
☒	ArrivalCity	Denver		
☒	Date	2022-12-08		

Body

Cookies

Headers (4)

Test Results (0/1)

500 INTERNAL SERVER ERROR

10 ms

449 B

Save Response

All

Passed

Skipped

Failed

FAIL

Status code is 200 | AssertionError: expected response to have status code 200 but got 500

La vérification est au statut **Fail**



Vérifier le contenu d'une balise Json

Objectif : Vérifier que la première balise FlightNumber a pour valeur 17105 est présent

Ajouter les Snippets **Response body : Contains String** et **Response body :Json value check**

The screenshot shows the Postman interface for a REST client. The request is a GET to `http://localhost:5000/flights_api/v1/resources/Flights?DepartureCity=Paris&ArrivalCity=Denver&Date=...`. The Tests tab is active, showing three test scripts:

```
1 pm.test("Le code retour est 200", function () {
2   pm.response.to.have.status(200);
3 });
4
5 pm.test("La réponse contient le tag FlightNumber", function () {
6   pm.expect(pm.response.text()).to.include("FlightNumber");
7 });
8
9 pm.test("Le premier vol est le 17105", function () {
10   var jsonData = pm.response.json();
11   pm.expect(jsonData[0].FlightNumber).to.eql(17105);
12 }
13
```

Annotations with red circles:

- 1: Status code: Code is 200
- 2: Response body: Contains string
- 3: Response body: JSON value check
- 4: Points to the first test script.
- 5: Points to the third test script.

The response body is shown in the Body tab, displaying a JSON array of flight data. The first flight (1^{er} vol) has a FlightNumber of 17105. The second flight (2^{ème} vol) has a FlightNumber of 17109.

Les libellés des tests ont été personnalisé

Vérifier le contenu d'une balise Json

Résultat :

The screenshot displays the Postman interface for a REST client. The top bar shows the title "Exercice - Ajout de tests / Recherche par ville de départ, d'arrivée et par date Copy" and a "Save" button. The main area is divided into several tabs: Params, Authorization, Headers (6), Body, Pre-request Script, Tests (selected), and Settings. The Tests tab contains three test scripts written in JavaScript, each preceded by a line number (1-12). The first test checks the status code (200), the second checks for the presence of the "FlightNumber" tag, and the third checks the value of the first flight number (17105). The right sidebar shows a "Send" button and a "Cookies" tab. Below the Tests tab, the "Test Results (3/3)" section is visible, showing three passed tests: "Le code retour est 200", "La réponse contient le tag FlightNumber", and "Le premier vol est le 17105". The bottom status bar indicates a successful response (200 OK) with a response time of 41 ms and a size of 1.4 KB.

Exercice - Ajout de tests / Recherche par ville de départ, d'arrivée et par date Copy

GET http://localhost:5000/flights_api/v1/resources/Flights?DepartureCity=Paris&ArrivalCity=Denver&Date=...

Params Authorization Headers (6) Body Pre-request Script Tests Settings Cookies

```
1 pm.test("Le code retour est 200", function () {
2   pm.response.to.have.status(200);
3 });
4
5 pm.test("La réponse contient le tag FlightNumber", function () {
6   pm.expect(pm.response.text()).to.include("FlightNumber");
7 });
8
9 pm.test("Le premier vol est le 17105", function () {
10   var jsonData = pm.response.json();
11   pm.expect(jsonData[0].FlightNumber).to.eql(17105);
12 }
```

Test scripts are written in JavaScript, and are run after the response is received. Learn more about [tests scripts](#)

Snippets

- Status code: Code is 200
- Response body: Contains string
- Response body: JSON value check
- Response body: Is equal to a string

Body Cookies Headers (4) Test Results (3/3) 200 OK 41 ms 1.4 KB Save Response

All Passed Skipped Failed

PASS Le code retour est 200

PASS La réponse contient le tag FlightNumber

PASS Le premier vol est le 17105

Synthèse des tests

Test	Description
<code>pm.response.to.have.status(200);</code>	Le statut http est 200
<code>pm.expect(pm.response.code).to.be.oneOf([201,202]);</code>	Le statut http est parmi ces valeurs
<code>pm.expect(pm.response.responseTime).to.be.below(100);</code>	Le temps de réponse en ms
<code>pm.response.to.have.header('X-Cache');</code>	Le header existe
<code>pm.expect(pm.response.headers.get('X-Cache')).to.eql('HIT');</code>	Le header a cette valeur
<code>pm.expect(pm.cookies.has('sessionId')).to.be.true;</code>	Le cookie existe
<code>pm.expect(pm.cookies.get('sessionId')).to.eql('ad3se3ss8sg7sg3');</code>	Le cookie a cette valeur
<code>pm.response.to.have.body("OK");</code> <code>pm.response.to.have.body({'success'=true});</code>	Le corps de la réponse contient une valeur
<code>pm.expect(pm.response.text()).to.include('Order placed.');</code>	Le texte de la réponse contient une valeur
<code>const response = pm.response.json();</code> <code>pm.expect(response.age).to.eql(30);</code>	Parser la réponse au format json et vérifier une valeur.

Exercices

1. Exercice 1 : Rechercher un vol
 - 1.1. Requête GetFlight : recherche par numéro
 - 1.2. Requête GetFlights : recherche par ville de départ, d'arrivée et par date
 - 1.3. Consulter le log d'exécution

2. Exercice 2 : Les tests
 - 2.1. Valider le code retour de la requête
 - 2.2. Vérifier la présence d'un texte dans la réponse
 - 2.3. Vérifier le contenu d'une balise JSON

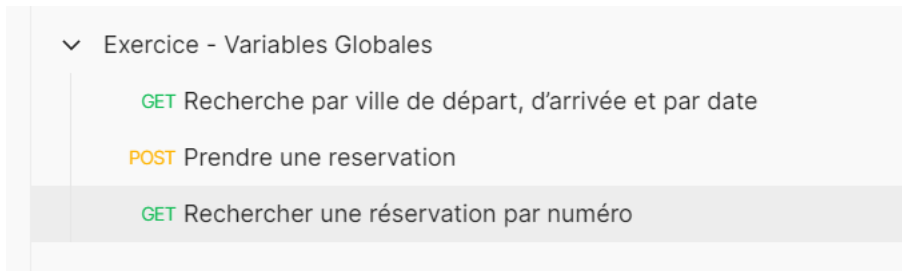
3. Enchaînement de plusieurs requêtes

Préparation d'une collection dédiée

Objectif: **Simuler le scénario d'utilisation suivant**

- Rechercher les vols disponibles avec une ville de départ, une ville d'arrivée et une date
Sélectionner le 1^{er} vol disponible
- Prendre une réservation sur ce vol
- Contrôler l'existence de la réservation

Création d'une nouvelle collection en dupliquant les requêtes suivantes



Enregistrer dans une variable globale

1^{er} requête : Reprendre la requête **Recherche par ville de départ, d'arrivée et par date**

Ajouter le snippet **Set a global variable**,
puis affecter la variable Numéro_de_vol avec la valeur du numéro de vol (jsonData[0].FlightNumber)

The screenshot displays the Postman interface for a REST client. The top bar shows the request method as GET and the URL as `http://localhost:5000/flights_api/v1/resources/Flights?DepartureCity=Paris&ArrivalCity=Denver&Date=...`. The 'Tests' tab is active, showing a JavaScript test script. The script includes a test for the response text containing 'FlightNumber' and another test that sets a global variable 'Numero_de_vol' to the first flight number in the response. The 'Test Results' tab at the bottom shows three passed tests: 'Le code retour est 200', 'La réponse contient le tag FlightNumber', and 'Le premier vol est le 17105 et la variable globale 'Numero_de_vol' est créée'. A red box highlights the 'Set a global variable' snippet in the 'Snippets' panel on the right.

```
5 pm.test("La réponse contient le tag FlightNumber", function () {
6   pm.expect(pm.response.text()).to.include("FlightNumber");
7 });
8
9 pm.test("Le premier vol est le 17105 et la variable globale
   'Numero_de_vol' est créée", function () {
10   var jsonData = pm.response.json();
11   pm.expect(jsonData[0].FlightNumber).to.eql(17105);
12
13   pm.globals.set("Numero_du_Vol", jsonData[0].FlightNumber);
14
15 });
```

Test Results (3/3)

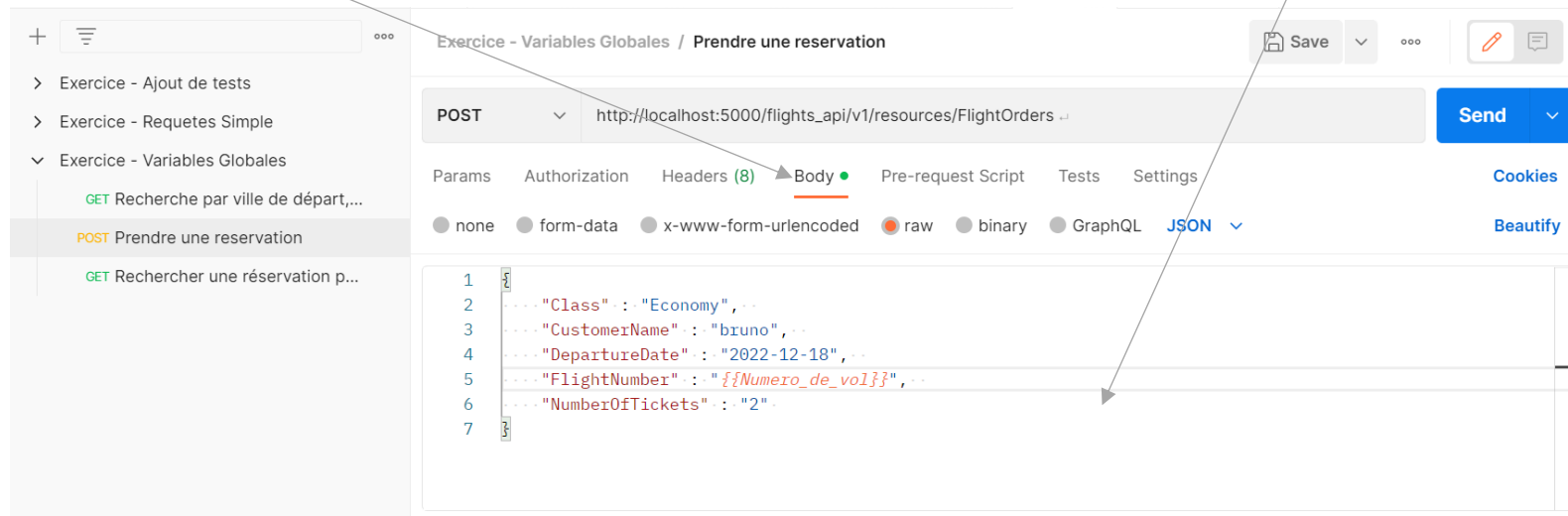
- PASS Le code retour est 200
- PASS La réponse contient le tag FlightNumber
- PASS Le premier vol est le 17105 et la variable globale 'Numero_de_vol' est créée

Note : Attention, déposer ce snippet dans **Response body:Json value check**, càd juste avant son accolade fermante

Utiliser la variable enregistrée

2^{ème} requête : Reprendre la requête **Prendre une reservation** et ajouter la variable globale

Sur l'onglet **Body**, positionner la variable globale **"{{Numero_de_vol}}"** pour le champ **FlightNumber**



Utiliser la variable enregistrée

2^{ème} requête : Sur la requête **Prendre une reservation**, enregistrer le numéro de réservation

Sur l'onglet Tests, ajouter quelques snippets (tests) et personnaliser-les, comme suit

Exercice - Variables Globales / Prendre une reservation

POST http://localhost:5000/flights_api/v1/resources/FlightOrders

Params Authorization Headers (8) Body Pre-request Script Tests Settings

```
1 pm.test("Le code retour est 200", function () {
2   pm.response.to.have.status(200);
3 });
4
5 pm.test("La réponse contient le tag Ordernumber", function () {
6   pm.expect(pm.response.text()).to.include("OrderNumber");
7 });
8
9 pm.test("Le numéro de réservation est supérieur à 80", function () {
10   var jsonData = pm.response.json();
11   pm.expect(jsonData.OrderNumber).to.be.greaterThan(80);
12
13   //Ce commentaire pour la nouvelle variable globale
14   pm.globals.set("Num_de_reservation", jsonData.OrderNumber);
15
16 });
17
```

Snippets

Status code: Code is 200

Response body: Contains string
(la réponse doit contenir le numéro d'ordre)

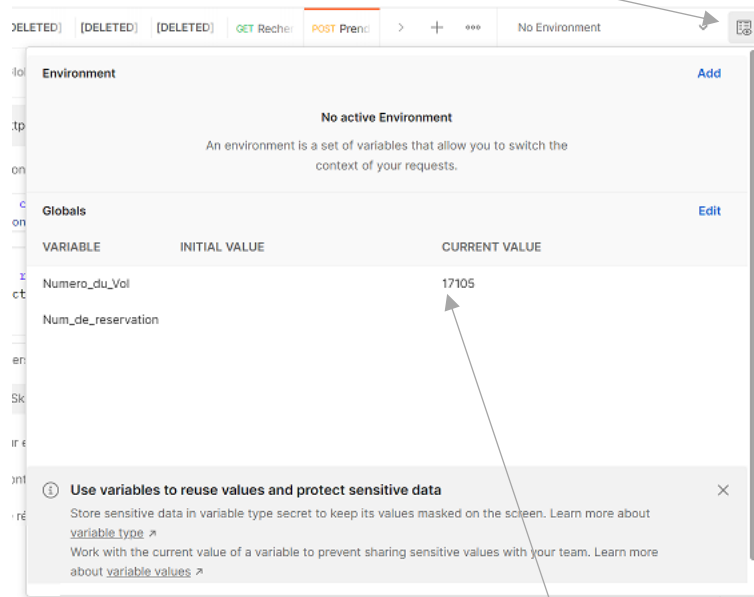
Response body: JSON value Check
Vérifier que le numéro d'ordre est plus grand que)

Set a global variable
(la nouvelle variable global est Num_de_reservation)

Utiliser la variable enregistrée

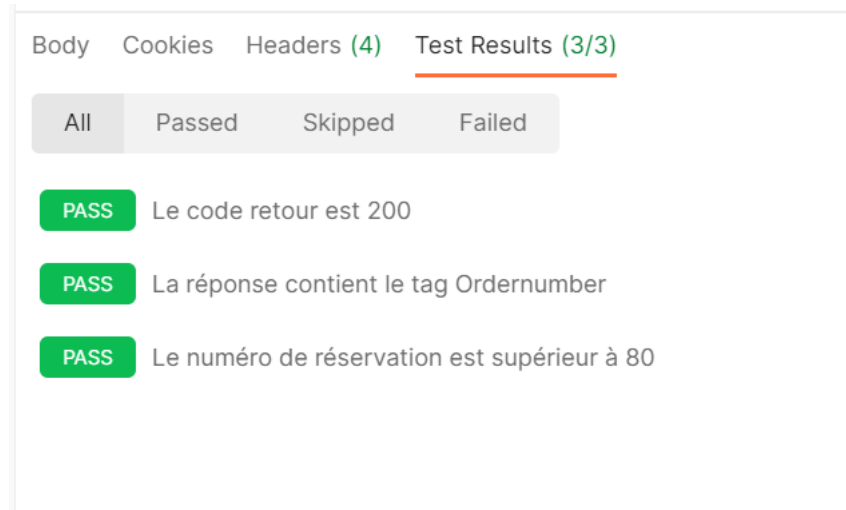
2^{ème} requête : Tester la requête **Prendre une reservation** seule

Cliquer sur l'icone Environnement
pour visualiser les variables globales



Vérifier la présence d'une valeur ou renseigner la (champ éditable)

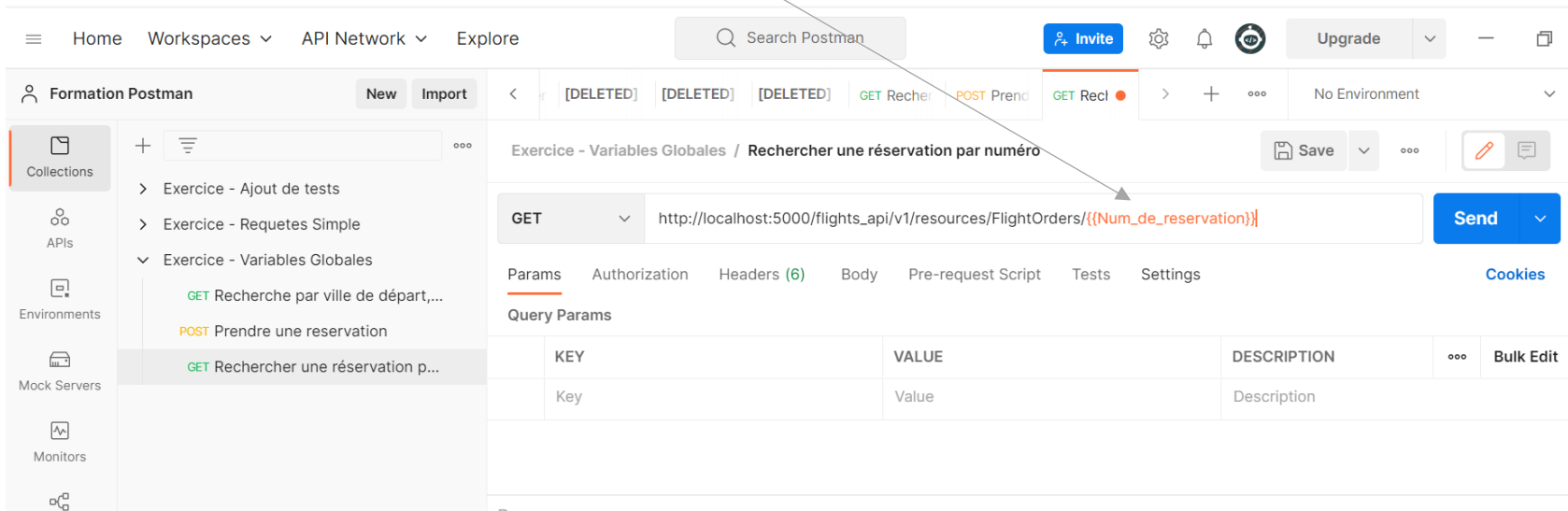
Lancer l'exécution de la requête



Utiliser la variable enregistrée

3^{ème} requête : Prendre la requête **Rechercher une réservation par numéro** et ajouter la variable globale

Ajouter la variable globale dans l'url de la requête

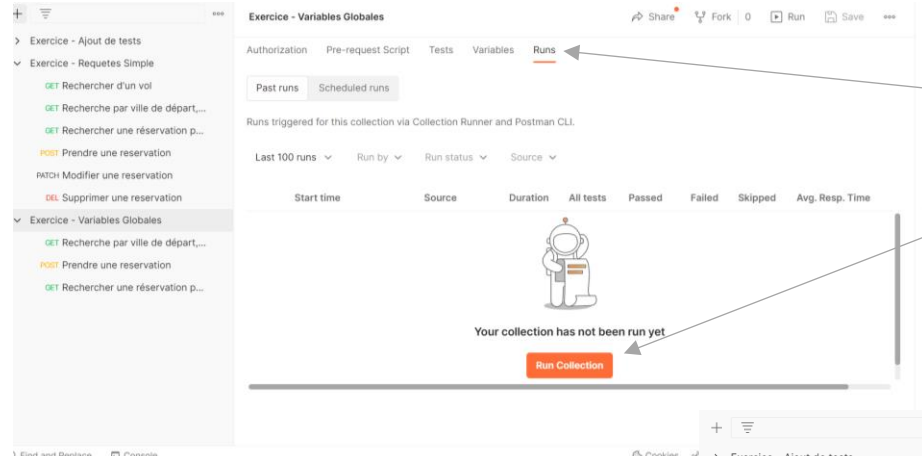


The screenshot shows the Postman interface. In the left sidebar, under 'Collections', the 'Exercice - Variables Globales' collection is expanded, and the request 'GET Rechercher une réservation p...' is selected. The main panel displays the details of this request. The URL is `http://localhost:5000/flights_api/v1/resources/FlightOrders/{{Num_de_reservation}}`. An arrow points from the text 'Ajouter la variable globale dans l'url de la requête' to the variable `{{Num_de_reservation}}` in the URL. The 'Query Params' table is empty.

KEY	VALUE	DESCRIPTION
Key	Value	Description

Utiliser la variable enregistrée

Exécuter l'enchaînement complet des tests

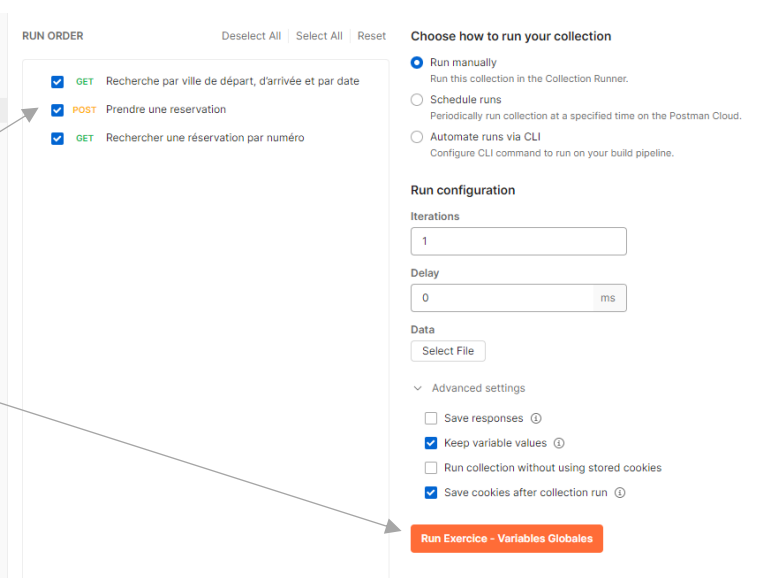


Sélectionner l'onglet **Runs** de la collection

Cliquer sur le bouton **Run Collection**

(Dé)Sélectionner les requêtes à exécuter

Cliquer sur le bouton Run Exercice – Variables Globales



RUN ORDER

- ☒ GET Recherche par ville de départ, d'arrivée et par date
- ☒ POST Prendre une reservation
- ☒ GET Rechercher une réservation par numéro

Choose how to run your collection

☒ Run manually
Run this collection in the Collection Runner.

☐ Schedule runs
Periodically run collection at a specified time on the Postman Cloud.

☐ Automate runs via CLI
Configure CLI command to run on your build pipeline.

Run configuration

Iterations: 1

Delay: 0 ms

Data: Select File

Advanced settings

- ☐ Save responses
- ☒ Keep variable values
- ☐ Run collection without using stored cookies
- ☒ Save cookies after collection run

Run Exercice - Variables Globales

Utiliser la variable enregistrée

Résultats de l'exécution

The screenshot displays the Postman interface with the 'Exercice - Variables Globales' collection selected. The 'Run' button is highlighted in orange. The 'Run results' section shows a summary table and a detailed list of test results.

Source	Environment	Iterations	Duration	All tests	Avg. Resp. Time
Runner	none	1	404ms	8	20 ms

All Tests Passed (8) Failed (0) Skipped (0) [View Summary](#)

- GET** Recherche par ville de départ... http://localhost:5000/flights_api/v1/resources/Flights?Departur... / Recher... 200 OK 13 ms 1.431 KB 1
- Pass Le code retour est 200
- Pass La réponse contient le tag FlightNumber
- Pass Le premier vol est le 17105 et la variable globale 'Numero_de_vol' est créée
- POST** Prendre une reservation http://localhost:5000/flights_api/v1/resources/FlightOrders / Prendre une reservat... 200 OK 39 ms 187 B
- Pass Le code retour est 200
- Pass La réponse contient le tag Ordernumber
- Pass Le numéro de réservation est supérieur à 80
- GET** Rechercher une réservation p... http://localhost:5000/flights_api/v1/resources/FlightOrders/{{Nu... / Recherc... 200 OK 8 ms 315 B
- Pass Le code retour est 200
- Pass OrderNumber est bien présent

Console (selected):

- ▶ GET http://localhost:5000/flights_api/v1/resources/Flights?DepartureCity=Paris&ArrivalCity=Denver&Date=2022-12-08 200 13 ms
- ▼ POST http://localhost:5000/flights_api/v1/resources/FlightOrders%0A 200 39 ms
- ▶ Network
- ▶ Request Headers
- ▶ Request Body ↗
- ▶ Response Headers
- ▶ Response Body ↗

- ▶ GET http://localhost:5000/flights_api/v1/resources/FlightOrders/87 200 8 ms

Annotations:

- An arrow points to the 'Console' tab in the bottom left.
- Text: 'Cliquer sur l'onglet **Console** Déplier l'arborescence des logs pour voir les détails'

Exercice 3 : Enchaîner plusieurs requêtes avec les variables globales

- 3. Exercice 3 : Enchaîner plusieurs requêtes avec les variables globales
 - 3.1. Enregistrer la donnée FlightNumber dans une variable globale
 - 3.2. Utiliser la variable dans la requête BookFlight et enregistrer le numéro de réservation
 - 3.3. Afficher le contenu des variables globales
 - 3.4. Exécuter l'enchaînement complet des tests

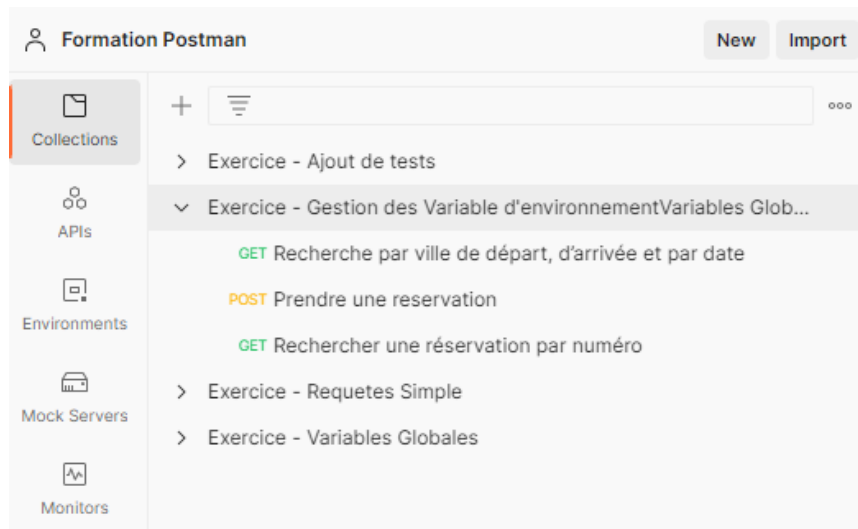
4. Utilisation des variables



Gestion des variables d'environnement

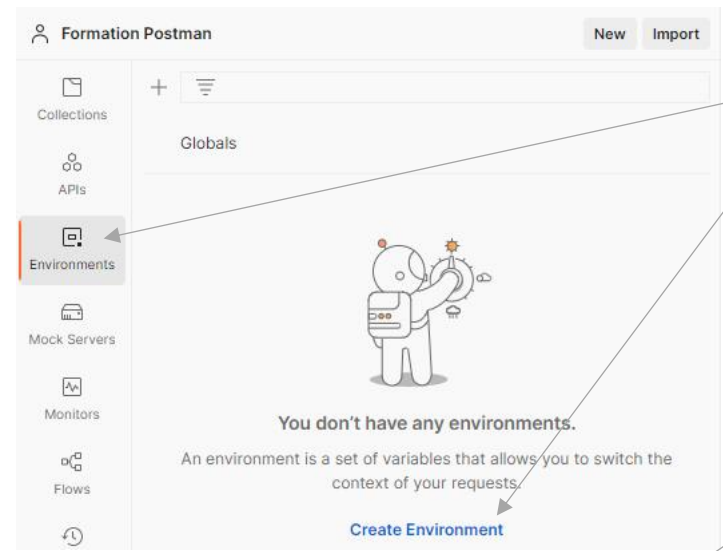
Objectif : Faciliter la maintenance des tests

Création d'une nouvelle collection en dupliquant la collection de l'exercice des variables Globales



Gestion des variables d'environnement

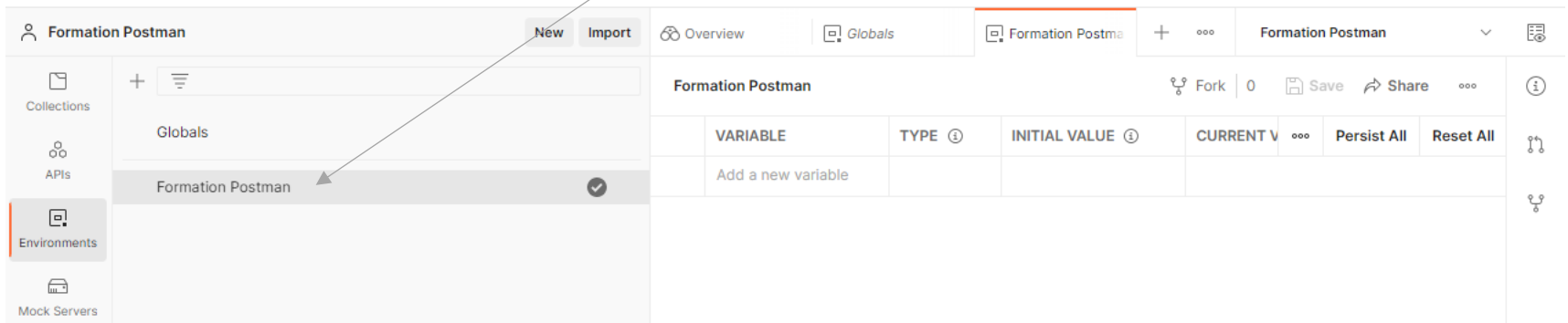
Création d'un environnement



Cliquer sur Environnement

Cliquer sur l'icône + ou sur le lien **Create Environment**

Renommer ce dossier pour le personnaliser



VARIABLE	TYPE ⓘ	INITIAL VALUE ⓘ	CURRENT V	...	Persist All	Reset All
Add a new variable						



Gestion des variables d'environnement

Création de nouvelle variable

Cliquer sur le lien **Add a new variable**

Saisir son nom et sa valeur

Sauvegarder

The screenshot shows the Postman interface with the 'Environments' tab selected. A table of environment variables is displayed with the following data:

VARIABLE	TYPE	INITIAL VALUE	CURRENT VALUE
✓ BaseURL	default	http://localhost:5000/fl...	http://localhost:5000/flights_api/v1/resour...
✓ Date	default	2022-12-25	2022-12-25
✓ Departure	default	Paris	Paris
✓ Arrival	default	London	London
Add a new variable			

Vérifier que l'environnement est actif

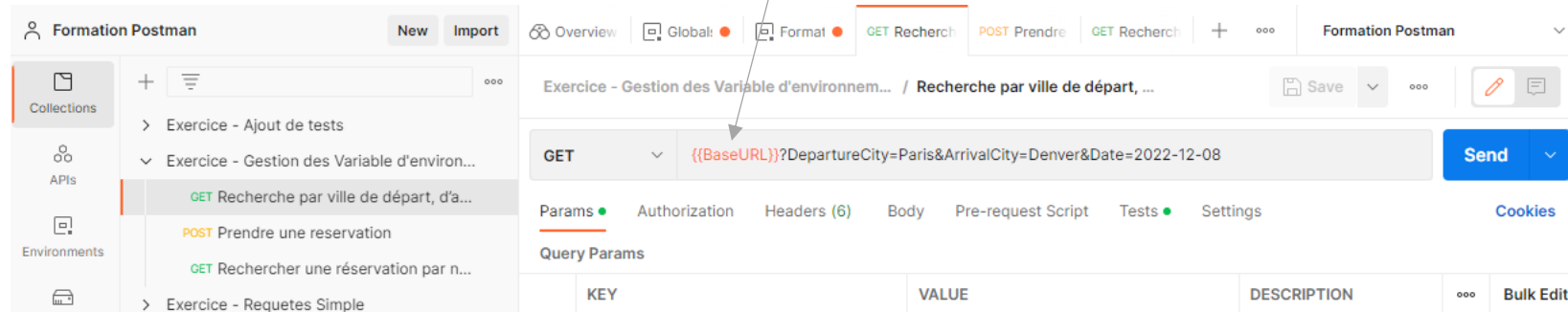
The screenshot shows a context menu for an environment. The menu options are:

- Set as active environment
- Move
- Export
- Duplicate (Ctrl+D)
- Create a fork (Ctrl+Alt+F)
- Create pull request

Gestion des variables d'environnement

Utilisation de la variable d'environnement

Remplacer http://localhost:5000/flights_api/v1/resources par `{{BaseURL}}` dans toutes les requêtes



En modifiant une seule donnée `{{BaseURL}}`, les requêtes seront envoyées à l'url d'un nouvel environnement

Gestion des variables d'environnement

Utilisation d'une variable d'environnement

The screenshot shows a REST client interface with a sidebar on the left containing a list of exercises. The main panel displays a GET request to `{{BaseURL}}/Flights?DepartureCity={{&ArrivalCity=Denver&Date=2022-12-08}}`. The 'Query Params' section is active, showing a table with columns KEY, VALUE, and DESCRIPTION. The 'DepartureCity' parameter is selected, and a dropdown menu is open, listing available environment variables: `Numero_du_Vol`, `Num_de_reservation`, `BaseURL`, `Date`, `Departure`, and `Arrival`. The dropdown also shows the current value of the selected variable, `17105`.

KEY	VALUE	DESCRIPTION
☒ DepartureCity	{{	
☒ ArrivalCity		
☒ Date		
Key		

Query Params

KEY VALUE DESCRIPTION Bulk Edit

☒ DepartureCity ☒ ArrivalCity ☒ Date

Key

Numero_du_Vol
Num_de_reservation
BaseURL
Date
Departure
Arrival

INITIAL
CURRENT 17105
SCOPE Global

Saisir {{ ,
une liste des variables
disponibles est proposée

The screenshot shows a REST client interface with a sidebar on the left. The main panel displays a POST request to `{{BaseURL}}/FlightOrders`. The 'Body' tab is active, showing a JSON payload. The 'DepartureDate' field is highlighted, and a dropdown menu is open, listing available environment variables: `Numero_du_Vol`, `Num_de_reservation`, `BaseURL`, `Date`, `Departure`, and `Arrival`. The dropdown also shows the current value of the selected variable, `17105`.

POST `{{BaseURL}}/FlightOrders` Send

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies Beautify

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "Class": "Economy",
3   "CustomerName": "bruno",
4   "DepartureDate": "{{Date}}",
5   "FlightNumber": "{{Numero_du_Vol}}",
6   "NumberOfTickets": "2"
7 }
```

Body Cookies Headers (4) Test Results (3/3) 200 OK 37 ms 187 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "OrderNumber": 92,
3   "TotalPrice": 278.94
4 }
```

Saisir les variables
entre double {{{}}

Gestion des variables d'environnement

Consultation des variables d'Environnement / Globales

Récupérer ici la liste des variables

Search Postman

Invite

Upgrade

<

[CONFLI]

Form

GET Recl

POST Prend

GET Recher

Exercic

>

+

...

Formation Postman

Globals

Global variables for a w
edited by anyone in tha

VARIABLE

☒ Numero_du_Vol

☒ Num_de_reserva

Add a new variat

Formation Postman

Edit

VARIABLE	INITIAL VALUE	CURRENT VALUE
BaseURL	http://localhost:5000/flights_api/v1/resource	http://localhost:5000/flights_api/v1/resourc es
Date	2022-12-25	2022-12-25
VilleDepart	Paris	Paris
VilleArrivee	London	London

Globals

Edit

VARIABLE	INITIAL VALUE	CURRENT VALUE
Numero_du_Vol		17105
Num_de_reservation		92



Exercice 4 : Variables d'environnement

- 4. Exercice 4 : Variables d'environnement
 - 4.1. Création des variables d'environnement
 - 4.2. Rendre les requêtes indépendantes de l'adresse des services Rest
 - 5.3. Utiliser les variables ville de départ, ville d'arrivée et date dans GetFlights
- 5.4. Modifier BookFlight en utilisant la variable Date
- 4.5. Exécuter la séquence et visualiser les variables

Synthèse des variables

Instruction	Description
<code>pm.globals.set('myVariable', MY_VALUE);</code>	Sauvegarder dans variable globale
<code>pm.globals.get('myVariable');</code>	Lire une variable globale
<code>pm.globals.unset('myVariable');</code>	Réinitialiser une variable globale
<code>pm.globals.clear();</code>	Effacer toutes les variables globales
<code>pm.collectionVariables.set('myVariable', MY_VALUE);</code> <code>pm.collectionVariables.get('myVariable');</code> <code>pm.collectionVariables.unset('myVariable');</code> <code>pm.environment.clear();</code>	Idem pour les variables de collection
<code>pm.environment.set('myVariable', MY_VALUE);</code> <code>pm.environment.get('myVariable');</code> <code>pm.environment.unset("myVariable");</code> <code>pm.environment.clear();</code>	Idem pour les variables d'environnement
<code>pm.variables.get('myVariable');</code>	Idem pour les variables de requête
<code>pm.environment.name</code>	Lire le nom de l'environnement actif
<code>pm.iterationData.get('myVariable');</code>	Lire une donnée sur itération avec un fichier CSV/JSON
<code>pm.variables.replaceIn('{{ \$randomFirstName }}');</code>	Utiliser une variable dynamique
<code>console.log(myVar);</code>	Ecrire dans la log d'exécution

Synthèse des variables dynamiques

Postman dispose de fonctions de génération dynamique des données pour les tests

La liste complète des possibilités : <https://learning.postman.com/docs/writing-scripts/script-references/variables-list/>

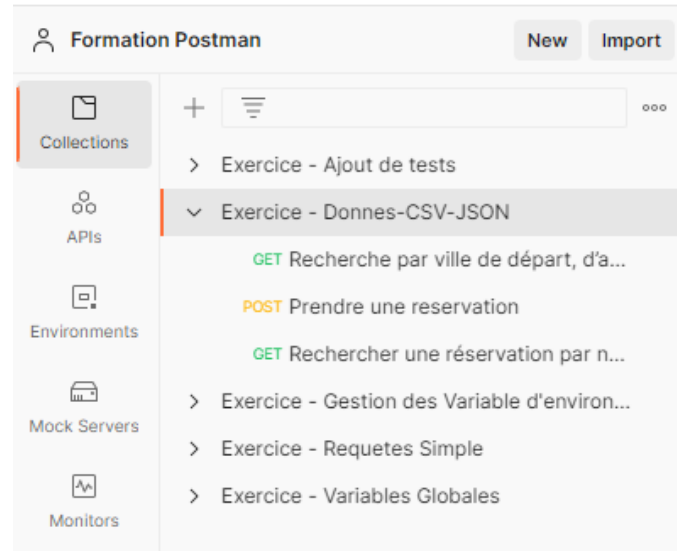
Variable dynamique	Description	Exempe
\$randomCity	City	East Ryanfurt, Jenkinsview
\$randomStreetName	Street name	Mckenna Pines, Schiller Highway, Vandervort Pike
\$randomStreetAddress	Street with number	98165 Tanya Passage, 0695 Monahan Squares
\$randomCountry	Country	Belgium, Antarctica (the territory South of 60 deg S)
\$randomCountryCode	Country code (2-letter)	GY, TK, BG
\$randomPrice	Price	244.00, 301.00
\$randomDatePast	Date in the past	Wed Mar 06 2019 04:17:52 GMT+0800 (WITA)
\$randomDateFuture	Date in the future	Wed Nov 20 2019 20:26:40 GMT+0800 (WITA)
\$randomDateRecent	Recent date	Thu Jun 20 2019 13:29:11 GMT+0800 (WITA)
\$randomCurrencyCode	Currency code	THB, HTG USD, AUD
\$randomCurrencyName	Currency name	Pound Sterling, Bulgarian Lev
\$randomPhrase	Phrase	We need to copy the online CSS microchip!
\$randomEmail	Email from popular email providers	Mable_Crist@hotmail.com, Ervin47@gmail.com
\$randomPassword	Password	v_Ptr4aTaBONsM0, 8xQM6pKgBUndK_J
\$randomLoremWord	Lorem ipsum word	ipsa, dolor, dicta
\$randomFirstName	First name	Dillan, Sedrick, Daniela
\$randomLastName	Last name	Schamberger, McCullough, Becker
\$randomPhoneNumber	Phone number	946.539.2542 x582, (681) 083-2162

5. Utilisation des données CSV / JSON

Utiliser les données dans un fichier CSV / JSON

Objectif : Automatiser une exécution d'une requête à partir de la lecture d'un fichier de données

Création d'une nouvelle collection en dupliquant la collection de l'exercice des variables d'environnement



Utiliser les données dans un fichier CSV / JSON

Préparation de la requête

Pour récupérer les données du fichier csv, la requête sera renseignée avec des variables portant le nom de la colonne à reprendre

	Departure, Arrival, Date, FlightNumber, Price, SeatsAvailable
1	Seattle, Portland, 2023-05-11, 1225, 138.47, 250
2	London, San Francisco, 2023-05-20, 17022, 171.8, 250
3	Paris, Denver, 2023-05-13, 17040, 163.8, 250
4	Seattle, San Francisco, 2023-05-18, 1226, 139.47, 250
5	London, Zurich, 2023-05-13, 11038, 159.8, 250
6	Frankfurt, Paris, 2023-05-07, 11225, 145.2, 250
7	Paris, Los Angeles, 2023-05-19, 17122, 156.47, 250
8	Sydney, London, 2023-05-12, 11765, 179.6, 250
9	Seattle, Denver, 2023-05-17, 1212, 125.47, 250
10	San Francisco, Paris, 2023-05-09, 20221, 112.2, 250
11	
12	

Exercice - Donnes-CSV-JS... / Recherche par ville de départ, d'arrivée et par d...

Save

GET

{{BaseURL}}/Flights?DepartureCity={{Departure}}&ArrivalCity={{Arrival}}&Date={{Date}}

Send

Params

Authorization

Headers (6)

Body

Pre-request Script

Tests

Settings

Cookies

Query Params

KEY	VALUE	DESCRIPTION		Bulk Edit
☒ DepartureCity	{{Departure}}			
☒ ArrivalCity	{{Arrival}}			
☒ Date	{{Date}}			
Key	Value	Description		

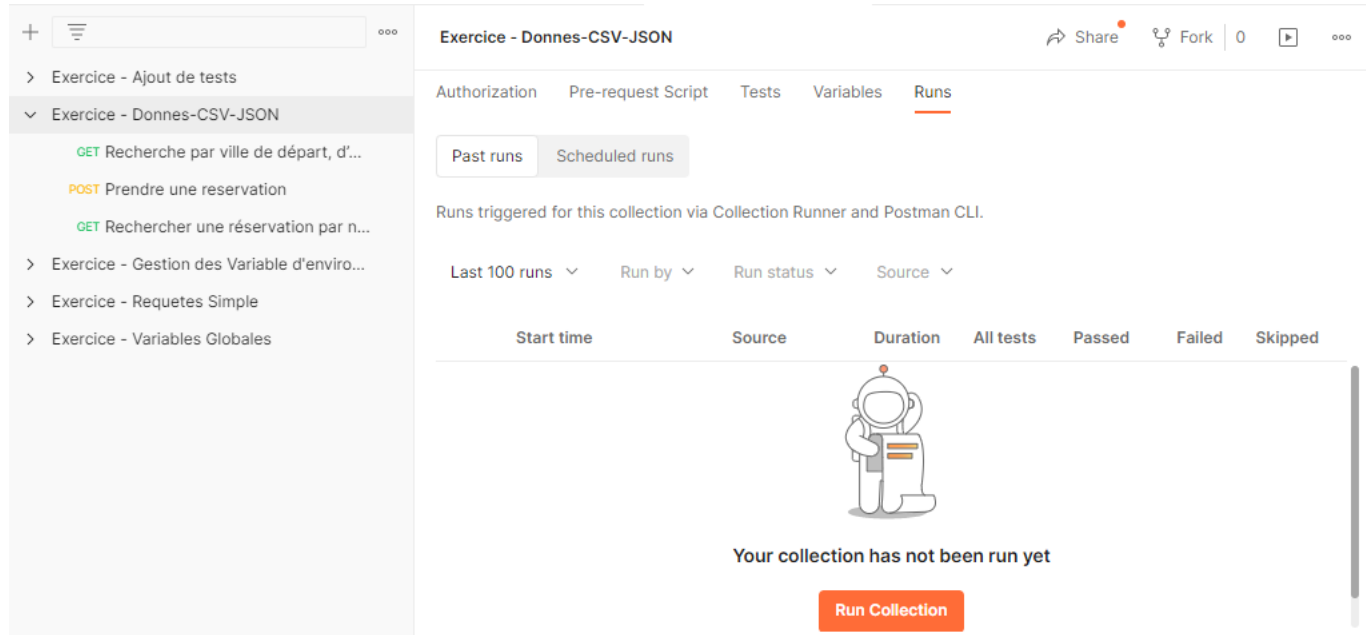


Utiliser les données dans un fichier CSV / JSON

Exécution de la requête

Sélectionner la collection **Exercice – Données-CSV-JSON**

Sur l'onglet **Run**, cliquer sur le bouton **Run Collection**



Utiliser les données dans un fichier CSV / JSON

Exécution de la requête

Cocher la requête prête (la première)

Au niveau du champ **Data**, cliquer sur le bouton **Select File**

Le fichier chargé peut être consulté, cela permet aussi de voir que le fichier est correctement pris en charge

Iteration	Departure	Arrival	Date	FlightNumber
1	"Seattle"	"Portland"	"2023-05-11"	1225
2	"London"	"San Francisco"	"2023-05-20"	17022
3	"Paris"	"Denver"	"2023-05-13"	17040
4	"Seattle"	"San Francisco"	"2023-05-18"	1226
5	"London"	"Zurich"	"2023-05-13"	11038
6	"Frankfurt"	"Paris"	"2023-05-07"	11225
7	"Paris"	"Los Angeles"	"2023-05-19"	17122
8	"Sydney"	"London"	"2023-05-12"	11765
9	"Seattle"	"Denver"	"2023-05-17"	1212
10	"San Francisco"	"Paris"	"2023-05-09"	20221

Cliquer sur le bouton **Run Exercice – Donnee-CSV-JSON**

Utiliser les données dans un fichier CSV / JSON

Résultats de la requête

The screenshot displays the Postman interface with the 'Exercice - Donnees-CSV-JSON - Run results' tab selected. The left sidebar shows the 'Collections' panel with the following structure:

- Exercice - Ajout de tests
 - GET Recherche par ville de départ, d'...
- Exercice - Donnees-CSV-JSON
 - GET Recherche par ville de départ, d'...
 - POST Prendre une reservation
 - GET Rechercher une reservation par n...
- Exercice - Gestion des Variable d'enviro...
- Exercice - Requetes Simple
- Exercice - Variables Globales

The main panel shows the 'Run results' for the selected test. It includes a table with the following data:

Source	Environment	Iterations	Duration	All tests	Avg. Resp. Time
Runner	Formation Postman	10	911ms	10	12 ms

Below the table, the 'All Tests' section shows the results for iterations 9 and 10. Both iterations passed, with a status of 'Pass' and a message 'Le code retour est 200'.

The bottom panel shows the 'Console' tab with the following log entries:

```
▶ GET http://localhost:5000/flights_api/v1/resources/Flights?DepartureCity=Paris&ArrivalCity=Los%20Angeles&Date=2023-05-19 200 | 12 ms
▶ GET http://localhost:5000/flights_api/v1/resources/Flights?DepartureCity=Sydney&ArrivalCity=London&Date=2023-05-12 200 | 11 ms
▶ GET http://localhost:5000/flights_api/v1/resources/Flights?DepartureCity=Seattle&ArrivalCity=Denver&Date=2023-05-17 200 | 13 ms
▼ GET http://localhost:5000/flights_api/v1/resources/Flights?DepartureCity=San%20Francisco&ArrivalCity=Paris&Date=2023-05-09 200 | 12 ms
  ▶ Network
  ▶ Request Headers
  ▶ Response Headers
  ▶ Response Body
    [{"Airline": "AA", "ArrivalCity": "Paris", "ArrivalTime": "02:23 PM", "DepartureCity": "San Francisco", "DepartureTime": "07:12 AM", "FlightNumber": 20210, "Price": 112.2, "SeatsAvailable": 250, "DayOfWeek": "Tuesday"}, {"Airline": "AA", "ArrivalCity": "Paris", "ArrivalTime": "06:23 PM", "DepartureCity": "San Francisco", "DepartureTime": "11:12 AM", "FlightNumber": 20221, "Price": 112.2, "SeatsAvailable": 250, "DayOfWeek": "Tuesday"}]
```

Détail de la réponse de la dernière requête

Utiliser les données dans un fichier CSV / JSON

Exécuter une séquence de test (avec deux requêtes)

The screenshot shows a REST client interface with a sidebar on the left containing a list of exercises. The main panel displays a GET request for a flight search endpoint. Below the request, the 'Tests' tab is active, showing a sequence of three tests. The first test checks the status code is 200. The second test checks that the response contains the 'FlightNumber' tag. The third test uses `pm.iterationData.get("FlightNumber")` to retrieve the flight number from a CSV file and compares it with the response. An arrow points from the `pm.iterationData.get` function in the test to a CSV file named 'Flights_data.csv' shown in a separate window below.

```
1 pm.test("Le code retour est 200", function () {
2   pm.response.to.have.status(200);
3 });
4
5 pm.test("La réponse contient le tag FlightNumber", function () {
6   pm.expect(pm.response.text()).to.include("FlightNumber");
7 });
8
9 pm.test("le numero de vol attendu dans la réponse " + pm.iterationData.get("FlightNumber"), function
10   () {
11     pm.expect(pm.response.text()).to.include(pm.iterationData.get("FlightNumber"));
12   });
```

Sur la requête **Recherche par ville de départ**

Ajouter la fonction **`pm.iterationData.get`** (« ») pour récupérer les données du fichier csv (utiliser le même nom)
Et pour pouvoir faire une comparaison avec les résultats de la réponse

The screenshot shows a CSV file named 'Flights_data.csv' with the following data:

1	Departure,Arrival,Date,FlightNumber,Price,SeatsAvailable
2	Seattle,Portland,2023-05-11,1225,138.47,250
3	London,San Francisco,2023-05-20,17022,171.8,250
4	Paris,Denver,2023-05-13,17040,163.8,250

Utiliser les données dans un fichier CSV / JSON

La deuxième requête, récupère **{{FlightNumber}}** du fichier CSV

The screenshot displays the Postman interface for an exercise titled "Exercice - Donnees-CSV-JSON / Prendre une reservation".

Request 1 (Top):

- Method: POST
- URL: `{{BaseURL}}/FlightOrders`
- Body Type: raw (JSON)
- Body Content:

```
1 {
2   "Class": "Economy",
3   "CustomerName": "bruno",
4   "DepartureDate": "{{Date}}",
5   "FlightNumber": "{{FlightNumber}}",
6   "NumberOfTickets": "2"
7 }
```

Request 2 (Bottom):

- Method: POST
- URL: `{{BaseURL}}/FlightOrders`
- Tests Tab Active:

```
1 pm.test("Vérifier le prix total", function () {
2   var jsonData = pm.response.json();
3   prix = pm.iterationData.get("Price");
4   pm.expect(jsonData.TotalPrice).to.eql(2*prix);
5 });
```

Ici, vérification que
le prix total (dans la réponse) = Nb de ticket * Prix (fichier CSV)
La fonction **`pm.iterationData.get`** (« ») est utilisée pour récupérer le prix des billets

Utiliser les données dans un fichier csv et JSON

Lancer l'enchaînement des requêtes

+

☰

...

> Exercice - Ajout de tests

> Exercice - Créer une boucle

▼ Exercice - Donnees-CSV-JSON

GET Recherche par ville de départ,...

POST Prendre une reservation

> Exercice - Gestion des Variable d'env...

> Exercice - Requetes Simple

> Exercice - Variables Globales

Exercice - Donnees-CSV-JSON - Run results

Run AgainAutomate Run ▼+ New RunExport Results

👤 Run on Today, 15:35:57 · [View all runs](#)

Source	Environment	Iterations	Duration	All tests	Avg. Resp. Time
Runner	Formation Postman	10	1s 727ms	40	17 ms

All TestsPassed (40)Failed (0)Skipped (0)[View Summary](#)

POST Prendre une reservation{{BaseURL}}/FlightOrders / Prendre une reservation200 OK37 ms188 B

PassVérifier le prix total

Iteration 10

GET Recherche par ville de départ, d'ar...{{BaseURL}}/Flights?DepartureCity={{Departure}}&ArrivalCit... / Recherc...200 OK6 ms585 B

PassLe code retour est 200

PassLa réponse contient le tag FlightNumber

Passle numero de vol attendu dans la réponse 20221

POST Prendre une reservation{{BaseURL}}/FlightOrders / Prendre une reservation200 OK15 ms187 B

PassVérifier le prix total

Find and ReplaceConsole

CookiesCapture requestsRunnerTrash🏠

Exercice 5 : Utiliser les données dans un fichier csv et JSON

- 5. Exercices 5 : Utiliser les données dans un fichier csv et JSON
 - 5.1. Utiliser les données dans un fichier csv
 - 5.2. Exécuter la séquence de test
 - 5.3. Utiliser les données dans un fichier json

6. Création d'une boucle

Création d'une boucle

Objectif : Répéter la même séquence de tests

Création d'une nouvelle collection avec les requêtes suivantes :

Rechercher une réservation par le nom du client
GET {{BaseURL}}/FlightOrders?CustomerName={{customer}}

Supprimer une réservation par numéro
DELETE {{BaseURL}}/FlightOrders/{order}

▼ Exercice - Créer une boucle

GET Rechercher une réservation par le nom du client

DEL Supprimer une reservation par numéro

Création d'une boucle

Objectif :

Récupérer la liste des réservations

Supprimer une réservation

Répéter la boucle si le nombre de réservation est supérieur à 3

Exercice - Créer une boucle / Rechercher une réservation par le nom du c...

GET `{{BaseURL}}/FlightOrders?CustomerName=bruno` Send

Params Auth Headers (6) Body Pre-req. Tests Settings Cookies

Query Params

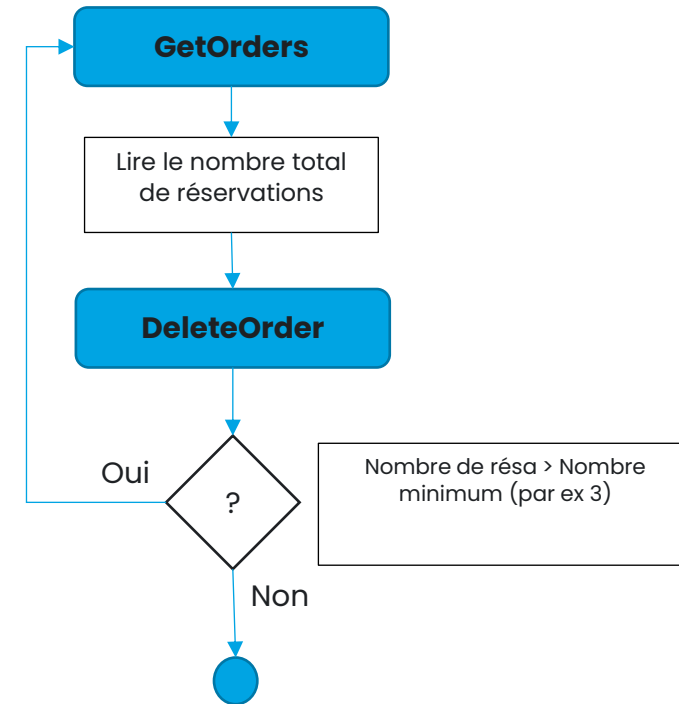
KEY	VALUE	DESCRIPTION
☒ CustomerName	bruno	

Body Cookies Headers (4) Test Results (1/1) 200 OK 8 ms 10.59 KB Save Response

Pretty Raw Preview Visualize

```
[{"Class": "First", "CustomerName": "bruno", "DepartureDate": "2022-12-18 18:15", "FlightNumber": 15113, "NumberOfTickets": 5, "OrderNumber": 82, "TotalPrice": 277.2}, {"Class": "Economy", "CustomerName": "bruno", "DepartureDate": "2022-12-18 08:00", "FlightNumber": 17105, "NumberOfTickets": 2, "OrderNumber": 83, "TotalPrice": 278.94}, {"Class": "Economy", "CustomerName": "bruno", "DepartureDate": "2022-12-18 08:00", "FlightNumber": 17105, "NumberOfTickets": 2, "OrderNumber": 84, "TotalPrice": 278.94}, {"Class": "Economy", "CustomerName": "bruno", "DepartureDate": "2022-12-18 08:00", "FlightNumber": 17105, "NumberOfTickets": 2, "OrderNumber": 85, "TotalPrice": 278.94}, {"Class": "Economy", "CustomerName": "bruno", "DepartureDate": "2022-12-18 08:00", "FlightNumber": 17105, "NumberOfTickets": 2, "OrderNumber": 86, "TotalPrice": 278.94}, {"Class": "Economy", "CustomerName": "bruno", "DepartureDate": "2022-12-18 08:00", "FlightNumber": 17105, "NumberOfTickets": 2, "OrderNumber": 87, "TotalPrice": 278.94}, {"Class": "Economy", "CustomerName": "bruno", "DepartureDate": "2022-12-18 08:00", "FlightNumber": 17105, "NumberOfTickets": 2, "OrderNumber": 88, "TotalPrice": 278.94}
```

Liste des réservations



Création d'une boucle

Préparation de la première requête : Récupérer la liste des réservations

+

☰

...

> Exercice - Ajout de tests

> Exercice - Créer une boucle

GET Rechercher une réservation p...

DEL Supprimer une reservation par...

> Exercice - Donnees-Csv-JSON

> Exercice - Gestion des Variable d'env...

> Exercice - Requetes Simple

> Exercice - Variables Globales

Exercice - Créer une boucle / Rechercher une réservation par le nom du client

Save

...

✎

💬

GET

{{BaseURL}}/FlightOrders?CustomerName=bruno

Send

Params

Authorization

Headers (6)

Body

Pre-request Script

Tests

Settings

Cookies

```
1 //Vérification du nom du client de la première reservation
2 pm.test("Vérifier le nom du client ", function () {
3     var jsonData = pm.response.json();
4     console.log(jsonData.length)
5     pm.expect(jsonData.length).to.be.gt(1)
6
7     pm.globals.set("Nb_Reservation", jsonData.length);
8 });
9
10 //Récupérer le nombre de reservation
11 pm.test("Vérifier qu'il existe au moins une réservation", function () {
12     var jsonData = pm.response.json()
13     pm.expect(jsonData[0].CustomerName).to.be.eq("bruno");
14 });
15
16 //Récupérer les numéro de la première reservation
17 pm.test("Vérifier que le numéro de réservation est supérieur à 80", function () {
18     var jsonData = pm.response.json();
19     pm.expect(jsonData[0].OrderNumber).to.be.greaterThan(80)
20
21     //Sauvegarde de la 1er reservation dans une variable globale Num_Reservation
22     pm.globals.set("Num_Reservation", jsonData[0].OrderNumber);
23 });
24
```

Informations récupérées après exécution:
Identifiant de la première réservation
Le nombre de réservation total

Où au niveau des logs

Globals		
VARIABLE	INITIAL VALUE	CURRENT VALUE
Num_Reservation		82
Nb_Reservation		63

☰ Online 🔍 Find and Replace 📄 Console

▶ GET http://localhost:5000/flights_api/v1/resources/FlightOrders?CustomerName=bruno

63



Création d'une boucle

Préparation de la deuxième requête : Supprimer une réservation

Récupération de la variable globale du numéro de réservation à supprimer

The screenshot shows the Postman interface with a collection named 'Formation Postman'. The active request is a DELETE request titled 'Supprimer une reservation par numéro'. The URL is configured as `{{BaseURL}}/FlightOrders/{{Num_Reservation}}`. The response is displayed in the 'Body' tab, showing a JSON object: `{ "true": "Order deleted 82 " }`. The status is 200 OK, 39 ms, 174 B.

Request Configuration:

- Method: DELETE
- URL: `{{BaseURL}}/FlightOrders/{{Num_Reservation}}`
- Query Params:

KEY	VALUE	DESCRIPTION
Key	Value	Description

Response:

```
1 {
2   "true": "Order deleted 82 "
3 }
```

Console Log:

```
DELETE http://localhost:5000/flights_api/v1/resources/FlightOrders/82 200 | 39 ms
```

Création d'une boucle

Préparation de la deuxième requête : Réaliser la boucle

Ajouter sur l'onglet **Pre-request Script** de la requête **Supprimer une réservation** :

1- Affichage du nombre total de réservation dans la console. Utiliser le snippet **Get global variable**

2- La boucle est réalisée avec la commande **postman.setNextRequest ("")**;

Avec le nom de la requête qui doit suivre : **Rechercher une réservation par le nom du client**

3- Ajouter une condition pour permettre cet enchainement (if condition { }).

The screenshot shows the Postman interface. On the left, a sidebar lists several exercises, with 'Exercice - Créer une boucle' expanded to show a list of requests. The selected request is 'DEL Supprimer une reservation par...'. The main panel displays the details of this request, which is a DELETE method to the endpoint `{{BaseURL}}/FlightOrders/{{Num_Reservation}}`. The 'Pre-request Script' tab is active, showing the following JavaScript code:

```
1 var Compteur_Reservation = pm.globals.get("Nb_Reservation");
2 console.log ("le nombre de reservation est : " + Compteur_Reservation);
3
4 if (Compteur_Reservation > 2) {
5   postman.setNextRequest ("Rechercher une réservation par le nom du client");
6 }
7
```

Création d'une boucle

Lancement de la boucle

Sur l'onglet **Runs**,
Cliquer sur le bouton **Schedule Runs**

Choisir **Run Manually**
Cliquer sur le bouton **Run Exercice – Créer une boucle**

Création d'une boucle

Résultats

Enchaînent des tests

Décompte au niveau de la console

The screenshot displays the Postman interface for a collection named "Exercice - Créer une boucle". The left sidebar shows the collection structure with tests like "GET Rechercher une réservation p...", "DELETE Supprimer une réservation par...", "Exercice - Données CSV-JSON", "Exercice - Gestion des Variables d'env...", "Exercice - Requêtes Simple", and "Exercice - Variables Globales".

The main panel shows the "Run results" for the "Exercice - Créer une boucle" test. A summary table provides an overview of the run:

Source	Environment	Iterations	Duration	All tests	Avg. Resp. Time
Runner	Formation Postman	1	9s 355ms	204	17 ms

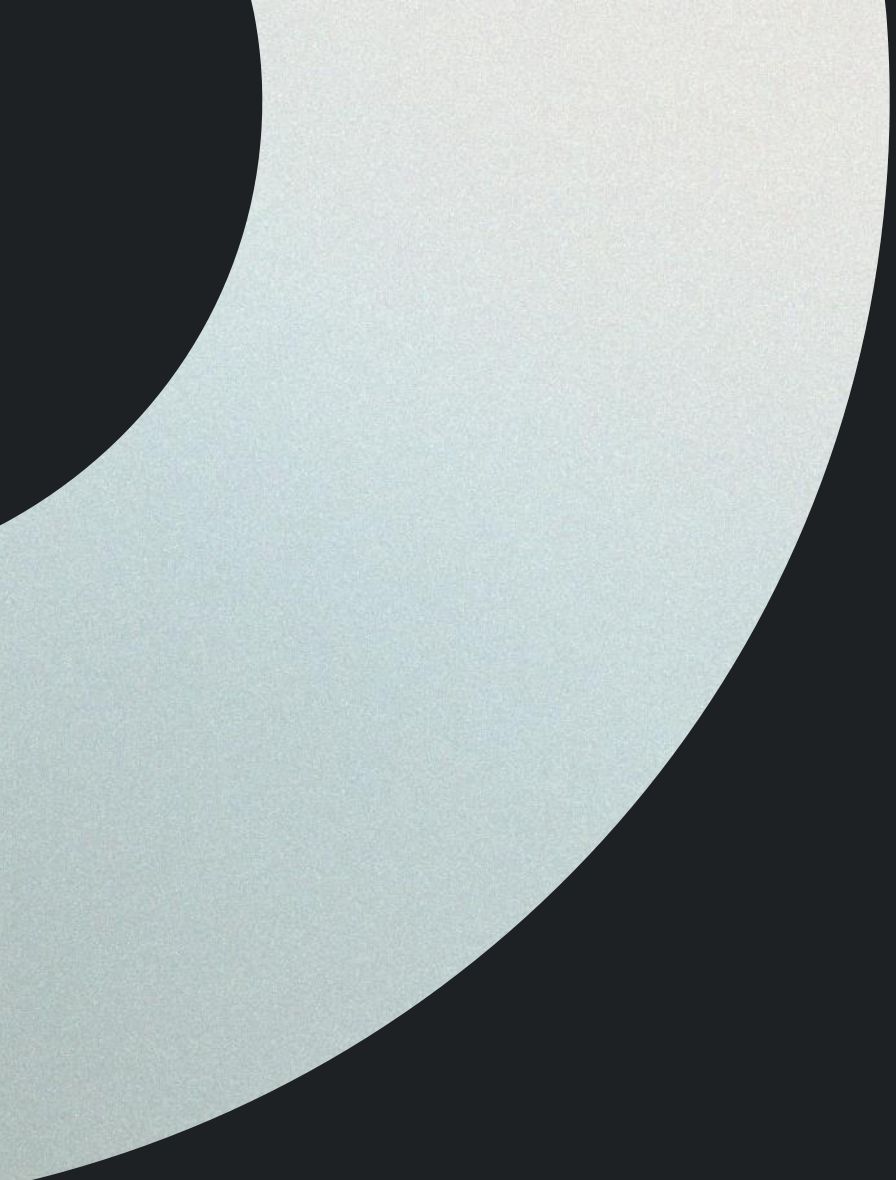
Below the summary, the "All Tests" section lists individual test results:

- Pass: Vérifier que le numéro de réservation est supérieur à 80
- DELETE: Supprimer une réservation par numéro. Status: 200 OK, 13 ms, 175 B.
- Pass: Le code retour est 200
- GET: Rechercher une réservation par le nom du client. Status: 200 OK, 6 ms, 487 B.
- Pass: Vérifier le nom du client
- Pass: Vérifier qu'il existe au moins une réservation
- Pass: Vérifier que le numéro de réservation est supérieur à 80
- DELETE: Supprimer une réservation par numéro. Status: 200 OK, 11 ms, 175 B.
- Pass: Le code retour est 200

The bottom console panel shows the execution logs, including the number of reservations and the results of the DELETE and GET requests. A bracket on the left indicates the "Décompte au niveau de la console" (Count at the console level).

Exercice 6 : Création d'une boucle

- 6. Exercice 6 : Créer une boucle
- 6.1. Lire le nombre total de réservation
- 6.2. Réaliser la boucle



6. Cas pratique

Gestion de projets Agile avec Taiga.io

Cas pratique

Taiga est un outil de gestion de projet pour les équipes agiles multifonctionnelles. Il dispose d'un ensemble riche de fonctionnalités, et il est très simple à utiliser grâce à son interface utilisateur intuitive.

Taiga dispose d'une API REST complète (celle utilisée par l'application Web) : <https://docs.taiga.io/api.html>

API à utiliser

Connexion	https://taigaio.github.io/taiga-doc/dist/api.html#auth-normal-login
Projets	https://taigaio.github.io/taiga-doc/dist/api.html#projects-list
Anomalies (Issues)	https://taigaio.github.io/taiga-doc/dist/api.html#issues-list

1. Créer un compte sur le site taiga.io (<https://tree.taiga.io/register>)
2. Créer un projet Scrum (vérifier que le module Issue est actif : Settings → Project → Modules)
3. Se connecter à l'application
4. Lire les informations du projet à partir de son nom
5. Récupérer la liste des anomalies (issues) du projet
6. Créer une anomalie et vérifier sur le site web que l'anomalie est bien créée
7. Afficher les informations de l'anomalie créée
8. Modifier le statut de l'anomalie
9. Créer une liste d'anomalie à partir d'un fichier csv

Merci

→ 16/03/2025